

Byzantine-tolerant detection of causality: there is no holy grail

Mechanisation in Isabelle/HOL of the impossibility results of
Misra and Kshemkalyani (Parallel Computing 124, 2025)

May 14, 2026

Abstract

We mechanise in Isabelle/HOL the impossibility results of Misra and Kshemkalyani, *Byzantine-tolerant detection of causality: there is no holy grail* (Parallel Computing 124 (2025), article 103136). The causality determination problem $CD(E, F, e_i^*)$ asks an asynchronous distributed algorithm to reconstruct the global execution history E at a correct process p_i , with respect to a target event e_i^* , such that the happened-before relation evaluated against the algorithm's collected F agrees with the truth in E . In the presence of even a single Byzantine process this is impossible.

We formalise:

- Theorems 1 and 2 (paper Section 4.1): no algorithm can prevent false negatives in the Byzantine model, and for internal events no algorithm can prevent both false negatives and false positives. Both proofs are constructive: given any candidate algorithm we exhibit a Byzantine adversary using a fresh natural number as event identifier.
- Theorems 3, 4, 5 (paper Section 4.2): the headline impossibility results for unicast, broadcast, and multicast communication. The proofs follow the paper's chain $\text{Consensus} \preceq \text{Black_Box} \preceq \text{CD}$, with the FLP impossibility imported from the AFP entry *FLP* (Bisping et al., 2025) as a proven theorem.

The development sits on top of the AFP entry *FLP*. Two named meta-level hypotheses are exposed on the headline theorems: `cd_can_identify_correct` capturing the paper's argument that solving CD requires identifying the correct set, and `bb_realizes_flp_consensus` capturing the standard reduction that an abstract Black_Box solver can be realised by an asynchronous distributed protocol. Neither is an internal HOL axiom; both are faithful to the paper's prose and discharged at interpretation time by the user.

License: BSD-3-Clause (matching the AFP *FLP* entry). The source paper is open access under CC-BY 4.0 and redistributed in this repository under that license.

Contents

1	Process identifiers, correct vs. Byzantine partition	5
2	Abstract Consensus solver signature	6
3	The Byzantine system locale	7

4	Events	7
5	Per-process and global histories	8
6	Program order and the happened-before relation	8
7	Causal past	10
8	Algorithm-perceived history	10
9	valid(F), false positives, false negatives	11
10	Adversary model	12
11	CD-solver signature	12
12	Communication mode: unicast / broadcast / multicast	14
13	The broadcast value w (paper, Section 4.2)	14
14	Black_Box output	16
15	Correctness of Black_Box	16
16	The trivial-execution gadget	17
17	Reduction R1: Consensus reduces to BlackBox	19
17.1	Agreement	19
17.2	Validity	20
17.3	Termination is implicit in totality	21
17.4	Composition: the headline reduction	22
18	Reduction R2: BlackBox reduces to CD	22
18.1	Constructive part: BB from an augmented CD solver	23
18.2	Three sub-claims mirroring <code>bb_correct_output</code>	24
19	Natural numbers used by an algorithm's output	26
20	Theorem 1: false negatives are unavoidable	28
21	Theorem 2: false negatives or false positives are unavoidable for internal events	33
22	FLP-style consensus solvability	41
23	BlackBox-to-FLP bridge predicate	42
24	Theorem 3: CD impossible under unicast	43
25	Theorem 4: CD impossible under broadcast	44

26	Theorem 5: CD impossible under multicast	45
27	Summary corollary	46
28	A pure-HOL “consensus solver” satisfying <code>solves_Consensus</code>	46
29	The natural-looking “abstract-consensus is unsolvable” claim is false	47

Reading order

The theories are intended to be read in the following order:

1. **ByzantineSystem**: the process partition (correct $\uplus$ Byzantine) and the abstract Consensus signature.
2. **Events**: events, per-process and global histories, program order, message order, happened-before relation (paper Definitions 1, 2).
3. **CD**: the Causality Determination problem, *valid*(F), false positives / negatives, the adversary model, the CD-solver signature (paper Definition 5).
4. **BlackBox**: the Black_Box problem of paper Section 4.2 with its piecewise broadcast value w .
5. **Reductions**: the two reductions of paper Section 4.2 – R1 (Consensus $\preceq$ BB) constructive in declarative Isar, R2 (BB $\preceq$ CD) constructive modulo the paper's named meta-level step `cd_can_identify_correct`.
6. **Theorems_1_2**: paper Section 4.1, Theorems 1 and 2, with explicit Byzantine adversary constructions using fresh natural numbers.
7. **FLP_Consensus**: the FLP-style consensus predicate with the impossibility proven against the AFP entry's `ConsensusFails`, and the bridge predicate `bb_realizes_flp_consensus`.
8. **Impossibility**: paper Section 4.2, Theorems 3, 4, 5 – the headline results.
9. **Foundation_Vacuity**: a regression diagnostic documenting why the abstract `solves_Consensus` predicate is too weak to express FLP, and motivating the FLP-style chain used by `Impossibility`.

```

theory ByzantineSystem
  imports
    FLP.AsynchronousSystem
    FLP.Execution
    FLP.FLPTheorem
begin

```

1 Process identifiers, correct vs. Byzantine partition

Paper, Section 2:

“The distributed system is modeled as an undirected graph $G = (P, C)$. Here P is the set of processes communicating asynchronously in the distributed system. Let $|P| = n$.”

The paper does not formalise the correct/Byzantine partition in the model itself; it talks about “a correct process p_c ” and “a Byzantine process p_b ” in prose. We make the partition explicit: `procs` is the set P , `correct` is the set of non-Byzantine processes, `byzantine` is the rest. Finiteness and non-emptiness of P are taken from the paper’s tacit assumptions.

```

locale process_partition =
  fixes
    procs      :: "'p set" and
    correct    :: "'p set" and
    byzantine  :: "'p set"
  assumes
    finite_procs:      "finite procs" and
    nonempty_procs:   "procs  $\neq$  {}" and
    partition_union:   "procs = correct  $\cup$  byzantine" and
    partition_disj:    "correct  $\cap$  byzantine = {}" and
    correct_subset:    "correct  $\subseteq$  procs" and
    byzantine_subset:  "byzantine  $\subseteq$  procs"

context process_partition
begin

lemma correct_finite [simp]: "finite correct"
  using finite_procs correct_subset by (rule rev_finite_subset)

lemma byzantine_finite [simp]: "finite byzantine"
  using finite_procs byzantine_subset by (rule rev_finite_subset)

lemma correct_byzantine_disjoint:
  assumes "p  $\in$  correct" shows "p  $\notin$  byzantine"
  using assms partition_disj by blast

lemma proc_correct_or_byz:
  assumes "p  $\in$  procs" shows "p  $\in$  correct  $\vee$  p  $\in$  byzantine"
  using assms partition_union by blast

```

end — context process_partition

2 Abstract Consensus solver signature

Paper, Section 4.2:

“In the Consensus problem, each process has an initial value and all correct processes must agree on a single value. The solution needs to satisfy the following three conditions.

- Agreement: All non-faulty processes must agree on the same single value.
- Validity: If all non-faulty processes have the same initial value, then the agreed-on value by all the non-faulty processes must be that same value.
- Termination: Each non-faulty process must eventually decide on a value.

”

We model a Consensus algorithm at the level of its observable input/output behaviour: it takes an initial-value vector V (boolean per process) and produces a decision (boolean per process).

Major deviation (load-bearing). At this level of abstraction the predicate `solves_Consensus` below only enforces Agreement and Validity, with Termination trivially holding by totality of the function type. The FLP impossibility result relies crucially on Termination-under-failure in an asynchronous distributed *protocol*, which a pure HOL function does not model; this makes `solves_Consensus` too weak for FLP to bite on, and is the reason the impossibility chain in `Impossibility.thy` routes through the FLP-style predicate in `FLP_Consensus.thy` instead. See `Foundation_Vacuity.thy` for a machine-checked counterexample to the naive “no abstract consensus solver exists” claim.

```
type_synonym 'p consensus_alg = "('p ⇒ bool) ⇒ 'p ⇒ bool"
```

```
definition consensus_agreement ::
```

```
"'p set ⇒ 'p consensus_alg ⇒ ('p ⇒ bool) ⇒ bool" where
"consensus_agreement C alg V ⟷ (∀p ∈ C. ∀q ∈ C. alg V p = alg V q)"
```

```
definition consensus_validity ::
```

```
"'p set ⇒ 'p consensus_alg ⇒ ('p ⇒ bool) ⇒ bool" where
"consensus_validity C alg V ⟷
  ((∀p ∈ C. V p) ⟶ (∀p ∈ C. alg V p)) ∧
  ((∀p ∈ C. ¬ V p) ⟶ (∀p ∈ C. ¬ alg V p))"
```

```
definition solves_Consensus ::
```

```
"'p set ⇒ 'p consensus_alg ⇒ bool" where
"solves_Consensus C alg ⟷
  (∀V. consensus_agreement C alg V ∧ consensus_validity C alg V)"
```

3 The Byzantine system locale

We bundle the process partition into a locale named `byzantineSystem`. The locale has no extra assumptions beyond those of `process_partition`: it merely fixes the partition `procs = correct ∪ byzantine`.

Historical note. Earlier versions of this development added a locale axiom `flp_consensus_impossibility` that claimed the FLP impossibility directly on `solves_Consensus`, but that axiom turned out to be unsatisfiable in HOL (the pure-HOL function `simple_alg C V p ≡ ∃q∈C. V q satisfies solves_Consensus`, so denying it collapses to `byzantine = {}`). See `Foundation_Vacuity.thy` for the machine-checked counter-example. The impossibility is now imported from the AFP entry’s `ConsensusFails` theorem via the FLP-style predicate in `FLP_Consensus.thy`, and the chain from CD-solvability to a contradiction is closed in `Impossibility.thy` through the `bb_realizes_flp_consensus` bridge.

```
locale byzantineSystem = process_partition procs correct byzantine
  for procs correct byzantine :: "'p set"
```

```
end
```

```
theory Events
  imports ByzantineSystem
begin
```

4 Events

The paper writes e_i^x for the x -th event of process p_i (Section 2: “Let e_i^x , where $x \geq 1$, denote the x th event executed by process p_i ”). We represent this as a tagged record carrying both the process p and the local sequence number n , plus – for sends and receives – the peer process and a message identifier that links the matching send and receive.

```
datatype 'p event =
  Internal (proc_of: 'p) (seq_of: nat)
| Send     (proc_of: 'p) (seq_of: nat) (peer: 'p) (msg_id: nat)
| Receive  (proc_of: 'p) (seq_of: nat) (peer: 'p) (msg_id: nat)
```

```
abbreviation is_send :: "'p event ⇒ bool" where
  "is_send e ≡ (∃p n q m. e = Send p n q m)"
```

```
abbreviation is_receive :: "'p event ⇒ bool" where
  "is_receive e ≡ (∃p n q m. e = Receive p n q m)"
```

```
abbreviation is_internal :: "'p event ⇒ bool" where
  "is_internal e ≡ (∃p n. e = Internal p n)"
```

A send and the corresponding receive of the same message are identified by a shared `msg_id`. The `matches` predicate also checks that the peer fields are dual (the send’s `peer` is the receiver, the receive’s `peer` is the sender).

Deviation: the paper does not bother to give `matches` a separate name; it is implicit in “the corresponding message receive event” (Definition 1). We name it because Isabelle’s

inductive happened-before definition needs it as a predicate.

```
fun matches :: "'p event  $\Rightarrow$  'p event  $\Rightarrow$  bool" where
  "matches (Send p n q m) (Receive q' n' p' m')  $\longleftrightarrow$  (p = p'  $\wedge$  q = q'  $\wedge$  m = m')"
| "matches _ _  $\longleftrightarrow$  False"
```

5 Per-process and global histories

Section 2 of the paper: “The execution history at p_i is the finite execution at p_i up to the current or most recent or specified local state.” Section 3: “Let E_i denote the actual execution history at $p_i \dots$ and let $E = \bigcup_i \{E_i\}$.” We represent both E_i (per process) and E (global) by a single mapping $'p \Rightarrow 'p \text{ event list}$: the per-process history is recovered by applying the mapping to a process, and the set of all events $T(E)$ is given by `events_of` below.

```
type_synonym 'p history_local = "'p event list"
type_synonym 'p history      = "'p  $\Rightarrow$  'p history_local"
```

$T(E)$ in the paper: the set of all events occurring in E , taken process-by-process and unioned.

```
definition events_of :: "'p history  $\Rightarrow$  'p event set" where
  "events_of H = ( $\bigcup$  p. set (H p))"
```

Well-formedness of a history: each per-process list (a) records only events whose `proc_of` field is that process, and (b) numbers its events 1, 2, 3, ... consecutively, matching the paper’s enumeration $\langle e_i^1, e_i^2, \dots \rangle$.

```
definition wf_history_local :: "'p  $\Rightarrow$  'p history_local  $\Rightarrow$  bool" where
  "wf_history_local p es  $\longleftrightarrow$ 
    ( $\forall e \in \text{set } es. \text{proc\_of } e = p$ )  $\wedge$ 
    ( $\forall k < \text{length } es. \text{seq\_of } (es ! k) = \text{Suc } k$ )"
```

```
definition wf_history :: "'p history  $\Rightarrow$  bool" where
  "wf_history H  $\longleftrightarrow$  ( $\forall p. \text{wf\_history\_local } p (H p)$ )"
```

```
lemma events_of_simp:
  "e  $\in$  events_of H  $\longleftrightarrow$  ( $\exists p. e \in \text{set } (H p)$ )"
by (auto simp: events_of_def)
```

6 Program order and the happened-before relation

Definition 1, rule 1 (Section 2):

“Program Order: For the sequence of events $\langle e_i^1, e_i^2, \dots \rangle$ executed by process p_i , $\forall x, y$ such that $x < y$ we have $e_i^x \rightarrow e_i^y$.”

We mechanise this as: e precedes e' in $H p$ iff e appears at a strictly earlier list index than e' in the same per-process list.

```
definition program_order :: "'p history  $\Rightarrow$  'p event  $\Rightarrow$  'p event  $\Rightarrow$  bool" where
  "program_order H e e'  $\longleftrightarrow$ 
    ( $\exists p i j. i < j \wedge j < \text{length } (H p) \wedge (H p) ! i = e \wedge (H p) ! j = e'$ )"
```

Definition 1, rule 2 (Section 2):

“Message Order: If event e_i^x is a message send event executed at process p_i and e_j^y is the corresponding message receive event at process p_j , then $e_i^x \rightarrow e_j^y$.”

Our `message_order` fires when the send and receive are matched by `matches` (same peer, same message id) and both events are recorded in the history.

Deviation: the paper takes for granted that the send-receive pair really happened; we make this explicit by requiring both endpoints to be in `events_of H`. For the algorithm-collected history F this is the right reading – if either endpoint is not in F , then “the algorithm doesn’t know about it”, and this is how the paper itself reasons when discussing false negatives (“Byzantine processes may delete information about e_h^x and m from F_h , leading to a false negative”, Section 4.2).

```
definition message_order :: "'p history  $\Rightarrow$  'p event  $\Rightarrow$  'p event  $\Rightarrow$  bool" where
  "message_order H e e'  $\longleftrightarrow$ 
    is_send e  $\wedge$  is_receive e'  $\wedge$  matches e e'  $\wedge$ 
    e  $\in$  events_of H  $\wedge$  e'  $\in$  events_of H"
```

One step of the happened-before relation: either a program-order step or a message-order step.

```
definition hb_step :: "'p history  $\Rightarrow$  'p event  $\Rightarrow$  'p event  $\Rightarrow$  bool" where
  "hb_step H e e'  $\longleftrightarrow$  program_order H e e'  $\vee$  message_order H e e'"
```

Definition 1, rule 3 (Section 2):

“Transitive Order: If $e \rightarrow e' \wedge e' \rightarrow e''$ then $e \rightarrow e''$.”

We define `hb` as the transitive closure of `hb_step`. This is the standard Lamport happened-before relation, evaluated against an arbitrary history H .

Note: the paper’s Definition 3 is the Byzantine-happened-before relation $\rightarrow_B$ that restricts both program order and message order to correct-process chains. We do not formalise $\rightarrow_B$ in this development – the impossibility theorems we cover (T1-T5) work on the plain $\rightarrow$. The `Send` and `Receive` constructors carry the peer field precisely so that a future development of Theorems 6-8 can define $\rightarrow_B$ by an inductive predicate quantifying over a correct set.

```
definition hb :: "'p history  $\Rightarrow$  'p event  $\Rightarrow$  'p event  $\Rightarrow$  bool" where
  "hb H e e'  $\longleftrightarrow$  (hb_step H)++ e e'"
```

The paper’s $e \rightarrow e' |_E$ notation (Section 3):

“Let $e_1 \rightarrow e_2 |_E$ and $e_1 \rightarrow e_2 |_F$ be the evaluation (1 or 0) of $e_1 \rightarrow e_2$ using E and F , respectively. ... If $e_h^x \notin T(E)$ then $e_h^x \rightarrow e_i^* |_E$ evaluates to false; ... If $e_h^x \notin T(F)$ (or $e_i^* \notin T(F)$) then $e_h^x \rightarrow e_i^* |_F$ evaluates to false.”

We mechanise this via a boolean `hb_eval` that requires both endpoints to be in `events_of H` before consulting the transitive closure.

```
definition hb_eval :: "'p history  $\Rightarrow$  'p event  $\Rightarrow$  'p event  $\Rightarrow$  bool" where
  "hb_eval H e e'  $\longleftrightarrow$ 
    (e  $\in$  events_of H  $\wedge$  e'  $\in$  events_of H  $\wedge$  hb H e e')"
```

7 Causal past

Definition 2 (Section 2): “The causal past of an event e is denoted as $CP(e)$ and defined as the set of events that causally precede e under $\rightarrow$.” We parameterise the definition by the history H against which the relation is evaluated – the paper writes simply $CP(e)$ because in Section 2 only the actual history E is in play.

```
definition causal_past :: "'p history  $\Rightarrow$  'p event  $\Rightarrow$  'p event set" where
  "causal_past H e = {e'. hb_eval H e' e}"
```

```
lemma causal_past_subset_events:
  "causal_past H e  $\subseteq$  events_of H"
by (auto simp: causal_past_def hb_eval_def)
```

8 Algorithm-perceived history

Section 3 of the paper distinguishes the actual global history E from the algorithm-collected history F :

“For any causality determination algorithm, let F_i be the execution history at p_i as perceived and collected by the algorithm and let $F = \bigcup_i \{F_i\}$ Byzantine processes may corrupt the collection of F to make it different from E Therefore it is not sufficient for the correct processes to agree mutually on a F that differs from E in what happened in E at the Byzantine processes; their F_j must also agree with E_j at all processes p_j .”

`correct_processes_agree` below is the property an algorithm *would* like to achieve at correct processes. Crucially, we do *not* bake it into admissibility: a Byzantine adversary can cause F to disagree with E even at correct processes (via contamination), so the impossibility argument needs to keep this as a separate property an algorithm might fail.

```
definition correct_processes_agree ::
  "'p set  $\Rightarrow$  'p history  $\Rightarrow$  'p history  $\Rightarrow$  bool" where
  "correct_processes_agree C E F  $\longleftrightarrow$  ( $\forall p \in C. F\ p = E\ p$ )"
```

An admissible pair (E, F) : both are well-formed histories. This is the weakest invariant we need; stronger constraints (the formal CD problem itself) appear in `CD.thy` via `valid`.

```
definition admissible_pair ::
  "'p set  $\Rightarrow$  'p history  $\Rightarrow$  'p history  $\Rightarrow$  bool" where
  "admissible_pair C E F  $\longleftrightarrow$  wf_history E  $\wedge$  wf_history F"
```

end

```
theory CD
  imports Events
begin
```

9 valid(F), false positives, false negatives

Definition 5 of the paper:

“The causality determination problem CD (E, F, e_i^{*}) for any event e_i^{*} ∈ T(E) at a correct process p_i is to devise an algorithm to collect the execution history E as F at p_i such that valid F = 1, where

$$valid(F) = \begin{cases} 1 & \text{if } \forall e_h^x, e_h^x \rightarrow e_i^*|_E = e_h^x \rightarrow e_i^*|_F \\ 0 & \text{otherwise} \end{cases}$$

”

The paper later clarifies (Section 4) that the universal quantifier ranges over T(E) ∪ T(F): “we have to evaluate ... even if e_h^x ∈ (T(E) ∪ T(F)) \ T(E) because such an e_h^x is recorded by the algorithm as part of F”. This is the inclusion of fake events the algorithm may have invented.

Faithfulness: we mechanise exactly this – the quantifier ranges over events_of E ∪ events_of F, not just events_of E.

definition valid :: "'p history ⇒ 'p history ⇒ 'p event ⇒ bool" where

```
"valid E F e_star <=>
  (∀ e ∈ events_of E ∪ events_of F.
    hb_eval E e e_star = hb_eval F e e_star)"
```

False negative (paper Section 4):

“∃ e_h^x such that e_h^x → e_i^{*}|_E = 1 ∧ e_h^x → e_i^{*}|_F = 0 (denoting a false negative, abbreviated FN).”

Note: the paper writes the disjunction “either FN or FP” when saying “0 is returned if one of the following two cases holds”. We define the two cases separately and prove valid iff neither occurs (lemma below).

definition false_negative ::

```
"'p history ⇒ 'p history ⇒ 'p event ⇒ bool" where
"false_negative E F e_star <=>
  (∃ e ∈ events_of E ∪ events_of F.
    hb_eval E e e_star ∧ ¬ hb_eval F e e_star)"
```

False positive (paper Section 4):

“∃ e_h^x such that e_h^x → e_i^{*}|_E = 0 ∧ e_h^x → e_i^{*}|_F = 1 (denoting a false positive, abbreviated FP).”

definition false_positive ::

```
"'p history ⇒ 'p history ⇒ 'p event ⇒ bool" where
"false_positive E F e_star <=>
  (∃ e ∈ events_of E ∪ events_of F.
    ¬ hb_eval E e e_star ∧ hb_eval F e e_star)"
```

lemma valid_iff_no_FP_FN:

```
shows "valid E F e_star
```

```

       $\longleftrightarrow \neg \text{false\_negative } E \ F \ e\_star$ 
       $\wedge \neg \text{false\_positive } E \ F \ e\_star$ "
proof -
  have "valid E F e_star
     $\longleftrightarrow (\forall e \in \text{events\_of } E \cup \text{events\_of } F.
      \text{hb\_eval } E \ e \ e\_star = \text{hb\_eval } F \ e \ e\_star)$ "
    by (simp add: valid_def)
  also have "...  $\longleftrightarrow \neg \text{false\_negative } E \ F \ e\_star$ 
     $\wedge \neg \text{false\_positive } E \ F \ e\_star$ "
    by (auto simp: false_negative_def false_positive_def)
  finally show ?thesis .
qed

```

10 Adversary model

In our model an adversary fixes the actual execution E , the process i at which the determination is to be made, and the target event e_star . The algorithm-collected F is *not* part of the adversary record – it is an output of the algorithm, computed from observable inputs (i, e_star) under the adversary’s chosen Byzantine strategy.

Deviation: the paper’s CD problem statement gives (E, F, e_i^*) as parameters of the problem. We separate roles so that “algorithm sees only its observable inputs” is an explicit modelling property, not buried in interpretation.

Section 3 of the paper:

“We assume that a correct process p_i needs to determine whether $e_h^x \rightarrow e_i^*$ holds and e_i^* is an event in $T(E)$.”

We mechanise this admissibility as: E is well-formed, i is correct (i.e. in C), e_star is an event at i , and e_star actually occurs in E .

```

record 'p adversary =
  adv_E      :: "'p history"
  adv_e_star :: "'p event"
  adv_i      :: 'p

definition adversary_admissible ::
  "'p set  $\Rightarrow$  'p adversary  $\Rightarrow$  bool" where
  "adversary_admissible C adv  $\longleftrightarrow$ 
    wf_history (adv_E adv)  $\wedge$ 
    adv_i adv  $\in C$   $\wedge$ 
    proc_of (adv_e_star adv) = adv_i adv  $\wedge$ 
    adv_e_star adv  $\in \text{events\_of } (\text{adv\_E } \text{adv})$ "

```

11 CD-solver signature

A CD-solver, given the inputs the algorithm at p_i can see – namely the process identifier i and the target event e_star – produces the collected history F it proposes plus a boolean “claim of validity”. Paper Section 4:

“When 1 is returned, the algorithm output matches God’s truth and solves CD correctly. Thus, returning 1 indicates that the problem has been solved

correctly by the algorithm using F ."

Deviation: the paper does not pin down whether the algorithm sees E during its computation. Our type signature commits to "algorithm sees only (i, e_star) ; F is its constructed view" – which is the strongest reading consistent with the paper's notion of F as "the execution history at p_i as perceived and collected by the algorithm" (Section 3).

```
type_synonym 'p cd_solver =
  "'p  $\Rightarrow$  'p event  $\Rightarrow$  'p history  $\times$  bool"
```

Two notions of correctness in increasing strength.

`solves_CD_decision` is the weaker form: the algorithm's boolean output agrees with *valid computed against the algorithm's own F'* . An algorithm satisfying this is internally consistent – it returns 1 iff its F' is a valid reconstruction of *some* execution.

`produces_valid_F` is the paper's intended reading: F' actually matches the adversary's true E in the sense of Definition 5. The impossibility theorems are stated about this stronger predicate; the lemma below shows it always entails the weaker form (so impossibility of one entails impossibility of the other).

```
definition solves_CD_decision ::
  "'p set  $\Rightarrow$  'p cd_solver  $\Rightarrow$  bool" where
  "solves_CD_decision C alg  $\longleftrightarrow$ 
    ( $\forall$ adv. adversary_admissible C adv  $\longrightarrow$ 
      (let (F', b) = alg (adv_i adv) (adv_e_star adv) in
        b = valid (adv_E adv) F' (adv_e_star adv)))"
```

```
definition produces_valid_F ::
  "'p set  $\Rightarrow$  'p cd_solver  $\Rightarrow$  bool" where
  "produces_valid_F C alg  $\longleftrightarrow$ 
    ( $\forall$ adv. adversary_admissible C adv  $\longrightarrow$ 
      (let (F', _) = alg (adv_i adv) (adv_e_star adv) in
        valid (adv_E adv) F' (adv_e_star adv)))"
```

From a `produces_valid_F` solver we trivially obtain a `solves_CD_decision` solver by always returning the boolean `True`: if F' is genuinely valid, then claiming "valid" is correct.

```
lemma produces_valid_F_implies_solves_CD_decision:
  assumes "produces_valid_F C alg"
  shows " $\exists$ alg'. solves_CD_decision C alg'"
proof -
  define alg' where
    "alg'  $\equiv$  ( $\lambda$ i e. (fst (alg i e), True))"
  have "solves_CD_decision C alg'"
  proof (unfold solves_CD_decision_def, intro allI impI)
    fix adv
    assume A: "adversary_admissible C adv"
    have "fst (alg' (adv_i adv) (adv_e_star adv))
      = fst (alg (adv_i adv) (adv_e_star adv))"
      by (simp add: alg'_def)
    moreover from A assms
    have "valid (adv_E adv) (fst (alg (adv_i adv) (adv_e_star adv)))"
```

```

      (adv_e_star adv)"
    by (auto simp: produces_valid_F_def Let_def split: prod.split)
ultimately show
  "(let (F', b) = alg' (adv_i adv) (adv_e_star adv) in
    b = valid (adv_E adv) F' (adv_e_star adv))"
  by (simp add: alg'_def Let_def)
qed
thus ?thesis by blast
qed

```

12 Communication mode: unicast / broadcast / multicast

Section 2 of the paper:

“There are three modes of communication: multicast, unicast, and broadcast. In multicast, a message is sent to a group G of processes corresponding to some subset of P . A unicast is a multicast where $|G| = 1$. A broadcast is a multicast where $G = P$.”

We carry the mode as a datatype tag.

Deviation: our `CD_solvable` predicate is *not* sensitive to the mode tag – it just existentially quantifies over algorithms producing valid F , and the `valid` predicate is mode-agnostic at the level of Definition 5. This is a deliberate simplification: at this abstraction the modes only differ in what an adversary may do (the adversary in broadcast mode commits to sending to all processes, etc.), not in the validity criterion itself. The three impossibility theorems are nevertheless stated separately so a richer development that refines the mode (e.g. adding BRB-as-layer for Theorem 4) can specialise the proof without re-stating the theorem.

```
datatype comm_mode = Unicast | Broadcast | Multicast
```

```
definition CD_solvable :: "comm_mode  $\Rightarrow$  'p set  $\Rightarrow$  bool" where
  "CD_solvable m C  $\longleftrightarrow$  ( $\exists$  alg. produces_valid_F C alg)"
```

```
end
```

```
theory BlackBox
  imports CD
begin
```

13 The broadcast value w (paper, Section 4.2)

Paper, Section 4.2, immediately after Theorem 3’s proof opens the BB definition:

“`Black_Box`(V, E, F, e_i^*) executed at p_i takes as input a vector V of initial boolean values, one per process, E, F , and local event e_i^* at a process p_i . `Black_Box` invoked at p_i acts as follows. The correct process p_i broadcasts

the value w where:

$$w = \begin{cases} 0 & \text{if each correct } p_j \text{ has } V[j] = 0 \\ 1 & \text{if each correct } p_j \text{ has } V[j] = 1 \\ CD(E, F, e_i^*) & \text{otherwise} \end{cases}$$

and locally returns L , a list of ids of correct processes.”

The CD output “`valid(F)`” is a boolean; we inline “`CD(E, F, e_i^*)`” as `valid` applied to the relevant arguments.

definition `w_value` ::

```
"'p set  $\Rightarrow$  ('p  $\Rightarrow$  bool)  $\Rightarrow$  'p history  $\Rightarrow$  'p history  $\Rightarrow$  'p event  $\Rightarrow$  bool" where
"w_value C V E F e_star =
  (if ( $\forall p \in C. \neg V p$ ) then False
   else if ( $\forall p \in C. V p$ ) then True
   else valid E F e_star)"
```

Three helper lemmas that decompose `w_value` by case – used both by the reduction R1 in `Reductions.thy` and by Validity arguments later.

lemma `w_value_uniform_false`:

```
assumes " $\forall p \in C. \neg V p$ "
shows "w_value C V E F e_star = False"
using assms unfolding w_value_def by simp
```

lemma `w_value_uniform_true`:

```
assumes nonempty: " $C \neq \{\}$ " and all_true: " $\forall p \in C. V p$ "
shows "w_value C V E F e_star = True"
```

proof -

```
from nonempty all_true have " $\neg (\forall p \in C. \neg V p)$ " by blast
with all_true show ?thesis unfolding w_value_def by simp
```

qed

lemma `w_value_mixed`:

```
assumes notF: " $\neg (\forall p \in C. \neg V p)$ " and notT: " $\neg (\forall p \in C. V p)$ "
shows "w_value C V E F e_star = valid E F e_star"
```

proof -

```
have step1: "(if ( $\forall p \in C. \neg V p$ ) then False
  else if ( $\forall p \in C. V p$ ) then True
  else valid E F e_star)
  = (if ( $\forall p \in C. V p$ ) then True else valid E F e_star)"
  using notF by (rule if_not_P)
have step2: "(if ( $\forall p \in C. V p$ ) then True else valid E F e_star)
  = valid E F e_star"
  using notT by (rule if_not_P)
have "w_value C V E F e_star =
  (if ( $\forall p \in C. \neg V p$ ) then False
   else if ( $\forall p \in C. V p$ ) then True
   else valid E F e_star)"
  by (simp add: w_value_def)
also have "... = (if ( $\forall p \in C. V p$ ) then True else valid E F e_star)"
  by (rule step1)
```

```

also have "... = valid E F e_star"
  by (rule step2)
finally show ?thesis .
qed

```

14 Black_Box output

A Black_Box solver at p_i outputs three things:

- the collected history F' (carried over from the embedded CD sub-invocation: the paper’s “else” branch of w reads CD (E, F, e_i^*));
- the broadcast value w (paper: “broadcasts the value w ”);
- the set L of correct-process ids (paper: “locally returns L , a list of ids of correct processes”).

Deviation: the paper uses a list, we use a set. This loses nothing for the impossibility – only the SET of correct ids matters when we later say “ $L = \text{correct}$ ” in the correctness condition. For the reduction R1, where the paper writes $\min L$, we hard-wire a single p_{star} instead; see `Reductions.thy`.

```

record 'p bb_output =
  bb_F :: "'p history"
  bb_w :: bool
  bb_L :: "'p set"

```

Algorithmic signature: given the BB inputs (P the full process set, V the initial-value vector, e_{star} the local target event, i the invoking process), produce a `bb_output`.

```

type_synonym 'p bb_solver =
  "'p set  $\Rightarrow$  ('p  $\Rightarrow$  bool)  $\Rightarrow$  'p event  $\Rightarrow$  'p  $\Rightarrow$  'p bb_output"

```

15 Correctness of Black_Box

Combining the paper’s three demands on a Black_Box output:

1. `bb_F out` is a valid reconstruction of the adversary’s `adv_E` (this is the embedded CD-correctness clause: “Solving Black_Box at p_i requires ... solving CD”, paper Section 4.2);
2. `bb_w out` equals `w_value` as paper-defined, evaluated at the algorithm’s own `bb_F out`;
3. `bb_L out` is exactly the set C of correct processes (paper: “Solving Black_Box at p_i requires identifying the set of correct processes”).

```

definition bb_correct_output ::
  "'p set  $\Rightarrow$  'p set  $\Rightarrow$  ('p  $\Rightarrow$  bool)
     $\Rightarrow$  'p history  $\Rightarrow$  'p event  $\Rightarrow$  'p
     $\Rightarrow$  'p bb_output  $\Rightarrow$  bool" where
  "bb_correct_output P C V E e_star i out  $\longleftrightarrow$ 

```

```

valid E (bb_F out) e_star ∧
bb_w out = w_value C V E (bb_F out) e_star ∧
bb_L out = C"

```

A BB solver *alg* *solves* the BB problem for *P*, *C* when, for every initial vector *V* and every admissible adversary, the output is correct in the sense above.

```

definition solves_BlackBox ::
  "'p set ⇒ 'p set ⇒ 'p bb_solver ⇒ bool" where
  "solves_BlackBox P C alg ⇔
    (∀ V adv. adversary_admissible C adv →
      bb_correct_output P C V (adv_E adv)
        (adv_e_star adv) (adv_i adv)
          (alg P V (adv_e_star adv) (adv_i adv)))"

```

```

definition BlackBox_solvable :: "'p set ⇒ 'p set ⇒ bool" where
  "BlackBox_solvable P C ⇔ (∃ alg. solves_BlackBox P C alg)"

```

end

```

theory Reductions
  imports BlackBox
begin

```

16 The trivial-execution gadget

To invoke the universal quantifier in *solves_BlackBox* and *produces_valid_F* (the validity-condition predicates over all admissible adversaries) we need at least one concrete admissible adversary to instantiate them at. This gadget supplies the simplest such adversary: a one-event history at a designated correct process *p_star*, with that single event playing the role of the target *e_star*.

This is a mechanisation artefact, not a step of the paper. Existing distributed-algorithm literature usually formulates BB as “runs correctly on the actual execution”; we need an “actual” execution the universal quantifier can range over, and the trivial one is sufficient for Validity.

```

definition trivial_history :: "'p ⇒ 'p history" where
  "trivial_history p_star ≡
    (λp. if p = p_star then [Internal p_star 1] else [])"

```

```

definition trivial_event :: "'p ⇒ 'p event" where
  "trivial_event p_star ≡ Internal p_star 1"

```

```

definition trivial_adversary :: "'p ⇒ 'p adversary" where
  "trivial_adversary p_star ≡
    (| adv_E = trivial_history p_star,
      adv_e_star = trivial_event p_star,
      adv_i = p_star |)"

```

```

lemma wf_history_local_one:
  shows "wf_history_local p_star [Internal p_star 1]"

```

```

    by (simp add: wf_history_local_def)

lemma wf_history_local_empty:
  shows "wf_history_local p []"
  by (simp add: wf_history_local_def)

lemma wf_history_trivial:
  shows "wf_history (trivial_history p_star)"
proof -
  have "\p. wf_history_local p (trivial_history p_star p)"
  proof -
    fix p
    show "wf_history_local p (trivial_history p_star p)"
    proof (cases "p = p_star")
      case True
      hence "trivial_history p_star p = [Internal p_star 1]"
        by (simp add: trivial_history_def)
      thus ?thesis
        using True wf_history_local_one by simp
    next
      case False
      hence "trivial_history p_star p = []"
        by (simp add: trivial_history_def)
      thus ?thesis
        using wf_history_local_empty by simp
    qed
  qed
  thus ?thesis by (simp add: wf_history_def)
qed

lemma events_of_trivial:
  shows "events_of (trivial_history p_star) = {Internal p_star 1}"
proof -
  have "events_of (trivial_history p_star)
    = ( $\bigcup$ p. set (trivial_history p_star p))"
    by (simp add: events_of_def)
  also have "... = {Internal p_star 1}"
    by (auto simp: trivial_history_def)
  finally show ?thesis .
qed

lemma adversary_admissible_trivial:
  assumes "p_star  $\in$  C"
  shows "adversary_admissible C (trivial_adversary p_star)"
proof -
  have eE: "adv_E (trivial_adversary p_star) = trivial_history p_star"
    by (simp add: trivial_adversary_def)
  have eI: "adv_i (trivial_adversary p_star) = p_star"
    by (simp add: trivial_adversary_def)
  have eS: "adv_e_star (trivial_adversary p_star) = trivial_event p_star"
    by (simp add: trivial_adversary_def)
  have "wf_history (adv_E (trivial_adversary p_star))"

```

```

    using wf_history_trivial eE by simp
  moreover have "adv_i (trivial_adversary p_star) ∈ C"
    using assms eI by simp
  moreover have
    "proc_of (adv_e_star (trivial_adversary p_star))
      = adv_i (trivial_adversary p_star)"
    by (simp add: eS eI trivial_event_def)
  moreover have
    "adv_e_star (trivial_adversary p_star)
      ∈ events_of (adv_E (trivial_adversary p_star))"
    by (simp add: eE eS events_of_trivial trivial_event_def)
  ultimately show ?thesis by (simp add: adversary_admissible_def)
qed

```

17 Reduction R1: Consensus reduces to BlackBox

Paper, Section 4.2 (proof of Theorem 3, step 2):

“Now we give the reduction from Consensus to Black_Box. To solve Consensus(V) at (a correct process) p_i , we invoke Black_Box(V, E, F, e_i^*) locally (and likewise to solve Consensus(V) at (each process) p_j , invoke Black_Box(V, E, F, e_j^*) at each p_j). Each correct process computes $\min(L)$ from the locally returned list L and outputs as its consensus value the broadcast value that it receives from $p_{\min(L)}$ and terminates. The conditions of Consensus – Agreement, Validity, and Termination – can be seen to be satisfied. So Consensus $\preceq$ Black_Box.”

Deviation: we do not compute $\min(L)$. Since BB’s correctness condition guarantees $\text{bb_L} = \text{correct}$ – the set is the same at every correct process – every correct process resolves $p_{\min(L)}$ to the same value, so any fixed correct p_{star} serves in place of $\min(L)$. We thread p_{star} as a constructor parameter.

```

definition consensus_from_bb ::
  "'p set ⇒ 'p ⇒ 'p bb_solver ⇒ 'p consensus_alg" where
  "consensus_from_bb P p_star bb_alg ≡
    (λV _. bb_w (bb_alg P V (trivial_event p_star) p_star))"

```

```

context byzantineSystem
begin

```

17.1 Agreement

Paper’s first Consensus property (Section 4.2):

“Agreement: All non-faulty processes must agree on the same single value.”

In our construction every correct process reads the same bb_w ($\text{bb_alg} \dots$) value (consensus_from_bb ignores its second argument). Agreement is immediate from this constancy.

```

lemma consensus_from_bb_agreement:
  fixes V :: "'p ⇒ bool"

```

```

shows "consensus_agreement correct (consensus_from_bb procs p_star bb_alg) V"
proof -
  have const_in_i:
    "\p q. consensus_from_bb procs p_star bb_alg V p
      = consensus_from_bb procs p_star bb_alg V q"
  by (simp add: consensus_from_bb_def)
  show ?thesis by (simp add: consensus_agreement_def const_in_i)
qed

```

17.2 Validity

Paper's second Consensus property (Section 4.2):

“Validity: If all non-faulty processes have the same initial value, then the agreed-on value by all the non-faulty processes must be that same value.”

We get Validity by instantiating `solves_BlackBox` at the trivial adversary and reading off the uniform-case branches of `w_value`: if all correct processes have $V[p] = \text{True}$, the BB output is the `w_value_uniform_true` branch, which is `True`; similarly for `False`.

```

lemma bb_w_at_trivial:
  assumes bb: "solves_BlackBox procs correct bb_alg"
    and p_star_C: "p_star ∈ correct"
  shows "bb_w (bb_alg procs V (trivial_event p_star) p_star)
    = w_value correct V
      (trivial_history p_star)
      (bb_F (bb_alg procs V (trivial_event p_star) p_star))
      (trivial_event p_star)"
proof -
  let ?adv = "trivial_adversary p_star"
  have adm: "adversary_admissible correct ?adv"
    using p_star_C by (rule adversary_admissible_trivial)
  from bb have univ:
    "∀V adv. adversary_admissible correct adv ⟶
      bb_correct_output procs correct V
        (adv_E adv) (adv_e_star adv) (adv_i adv)
        (bb_alg procs V (adv_e_star adv) (adv_i adv))"
  by (simp add: solves_BlackBox_def)
  from univ adm have bb_inst:
    "bb_correct_output procs correct V
      (adv_E ?adv) (adv_e_star ?adv) (adv_i ?adv)
      (bb_alg procs V (adv_e_star ?adv) (adv_i ?adv))"
  by blast
  hence
    "bb_w (bb_alg procs V (adv_e_star ?adv) (adv_i ?adv))
      = w_value correct V (adv_E ?adv)
        (bb_F (bb_alg procs V (adv_e_star ?adv) (adv_i ?adv)))
        (adv_e_star ?adv)"
  by (simp add: bb_correct_output_def)
  thus ?thesis
    by (simp add: trivial_adversary_def)
qed

```

```

lemma consensus_from_bb_validity:
  assumes bb: "solves_BlackBox procs correct bb_alg"
    and p_star_C: "p_star ∈ correct"
  shows "consensus_validity correct (consensus_from_bb procs p_star bb_alg) V"
proof -
  let ?out = "bb_alg procs V (trivial_event p_star) p_star"
  have C_ne: "correct ≠ {}" using p_star_C by blast
  have bb_w_eq:
    "bb_w ?out
     = w_value correct V (trivial_history p_star)
       (bb_F ?out) (trivial_event p_star)"
    by (rule bb_w_at_trivial[OF bb p_star_C])

  have case_True:
    "(∀p ∈ correct. V p)
     → (∀p ∈ correct. (consensus_from_bb procs p_star bb_alg) V p)"
  proof
    assume H: "∀p ∈ correct. V p"
    have "bb_w ?out = True"
      using bb_w_eq w_value_uniform_true[OF C_ne H] by simp
    thus "∀p ∈ correct. (consensus_from_bb procs p_star bb_alg) V p"
      by (simp add: consensus_from_bb_def)
    qed

  have case_False:
    "(∀p ∈ correct. ¬ V p)
     → (∀p ∈ correct. ¬ (consensus_from_bb procs p_star bb_alg) V p)"
  proof
    assume H: "∀p ∈ correct. ¬ V p"
    have "bb_w ?out = False"
      using bb_w_eq w_value_uniform_false[OF H] by simp
    thus "∀p ∈ correct. ¬ (consensus_from_bb procs p_star bb_alg) V p"
      by (simp add: consensus_from_bb_def)
    qed

  from case_True case_False
  show ?thesis by (simp add: consensus_validity_def)
qed

```

17.3 Termination is implicit in totality

Paper's third Consensus property (Section 4.2):

“Termination: Each non-faulty process must eventually decide on a value.”

Deviation (a load-bearing one): our 'p consensus_alg is a total HOL function, so every correct process *always* returns a decision and Termination holds trivially. This trivialisation of Termination is exactly what makes the abstract solves_Consensus predicate too weak to express FLP – see Foundation_Vacuity.thy – and is the reason the impossibility chain in Impossibility.thy goes through the FLP-style flp_consensus_solvable predicate of FLP_Consensus.thy rather than directly through solves_Consensus.

17.4 Composition: the headline reduction

```

theorem consensus_reduces_to_blackbox:
  assumes p_star_C: "p_star ∈ correct"
    and bb: "solves_BlackBox procs correct bb_alg"
  shows "solves_Consensus correct (consensus_from_bb procs p_star bb_alg)"
proof -
  have agr: "∧V. consensus_agreement correct
              (consensus_from_bb procs p_star bb_alg) V"
    by (rule consensus_from_bb_agreement)
  have val: "∧V. consensus_validity correct
              (consensus_from_bb procs p_star bb_alg) V"
    by (rule consensus_from_bb_validity[OF bb p_star_C])
  show ?thesis using agr val by (simp add: solves_Consensus_def)
qed

corollary BlackBox_solvable_imp_Consensus_solvable:
  assumes ne: "correct ≠ {}" and bb: "BlackBox_solvable procs correct"
  shows "∃alg. solves_Consensus correct alg"
proof -
  obtain bb_alg where solver: "solves_BlackBox procs correct bb_alg"
    using bb by (auto simp: BlackBox_solvable_def)
  obtain p_star where pc: "p_star ∈ correct" using ne by blast
  have "solves_Consensus correct (consensus_from_bb procs p_star bb_alg)"
    by (rule consensus_reduces_to_blackbox[OF pc solver])
  thus ?thesis by blast
qed

end — context byzantineSystem

```

18 Reduction R2: BlackBox reduces to CD

Paper, Section 4.2 (proof of Theorem 3, step 1):

“If there were an algorithm to make F match E, it requires identifying whether each of the processes that input their execution histories is correct or Byzantine, and tracing and dealing with / resolving the impact of contamination via message passing by the Byzantine processes from and through those Byzantine processes on the execution histories of processes at other processes. Thus, $\text{Black_Box} \preceq \text{CD}$.”

Deviation – meta-level step. The paper does not exhibit a syntactic construction of a `Black_Box` solver from a CD solver; the argument is meta-level (“it requires identifying ...”). We capture this faithfully as a single named locale assumption `cd_can_identify_correct` in the sub-locale `byzantineSystem_with_identification`:

“If there is an algorithm `cd_alg` that produces a valid F, then there is an algorithm `cd_alg'` that (i) produces the same valid F, (ii) returns the decision `True`, and (iii) also returns the set of correct processes.”

This is the positive form of the paper’s contrapositive (“producing valid F is impossible without internally identifying the correct set”). From this assumption the rest of

R2 is fully constructive: the BB solver projects the augmented CD solver's L field into bb_L, its F field into bb_F, and applies the piecewise definition of w_value to obtain bb_w.

```
context byzantineSystem
begin
```

```
type__synonym 'q cd_solver_with_L =
  "'q  $\Rightarrow$  'q event  $\Rightarrow$  'q history  $\times$  bool  $\times$  'q set"
```

The augmented predicate insists on the three properties Misra–Kshemkalyani actually need from the meta-level step:

1. the collected F is valid in the sense of Definition 5;
2. the algorithm's claim b is True, matching Definition 5's "returning 1 indicates that the problem has been solved correctly";
3. the reported list L equals the correct set.

```
definition produces_valid_F_with_L ::
  "'p set  $\Rightarrow$  'p cd_solver_with_L  $\Rightarrow$  bool" where
  "produces_valid_F_with_L C alg  $\longleftrightarrow$ 
    ( $\forall$  adv. adversary_admissible C adv  $\longrightarrow$ 
      (let (F', b, L) = alg (adv_i adv) (adv_e_star adv) in
        valid (adv_E adv) F' (adv_e_star adv)
           $\wedge$  b
           $\wedge$  L = C))"
```

```
end — context byzantineSystem
```

```
locale byzantineSystem_with_identification = byzantineSystem +
  assumes cd_can_identify_correct:
    "produces_valid_F correct cd_alg  $\implies$ 
       $\exists$  cd_alg'. produces_valid_F_with_L correct cd_alg'
         $\wedge$  ( $\forall$  i e. fst (cd_alg' i e) = fst (cd_alg i e))"
```

```
context byzantineSystem_with_identification
begin
```

18.1 Constructive part: BB from an augmented CD solver

Given an augmented CD solver (one that, by `produces_valid_F correct ?cd_alg  $\implies \exists$  cd_alg'. produces_valid_F_with_L correct cd_alg'  $\wedge$  ( $\forall$  i e. fst (cd_alg' i e) = fst (?cd_alg i e))`, also reports the correct set), we build a Black_Box solver by simple projection:

- `bb_F` := the augmented CD solver's collected F';
- `bb_L` := the augmented CD solver's reported L (which equals `correct` by assumption);
- `bb_w` := the paper's piecewise `w_value`, computed against L and the CD solver's boolean.

This matches the paper’s reduction in Section 4.2: “Solving Black_Box at p_i requires identifying the set of correct processes and solving CD.” The augmented CD solver does both; the projection just exposes its outputs in the BB record shape.

```
definition bb_from_cd_with_L ::
  "'p cd_solver_with_L  $\Rightarrow$  'p bb_solver" where
  "bb_from_cd_with_L cd_alg'  $\equiv$ 
    ( $\lambda P V e\_star i.$ 
      let (F', b, L) = cd_alg' i e_star in
      ( $\llcorner$  bb_F = F',
        bb_w = (if ( $\forall p \in L. \neg V p$ ) then False
                  else if ( $\forall p \in L. V p$ ) then True
                  else b),
        bb_L = L  $\llcorner$ )")
```

18.2 Three sub-claims mirroring bb_correct_output

```
lemma bb_from_cd_with_L_correct:
  assumes augmented: "produces_valid_F_with_L correct cd_alg'"
  shows "solves_BlackBox procs correct (bb_from_cd_with_L cd_alg')"
proof (unfold solves_BlackBox_def, intro allI impI)
  fix V adv
  assume adm: "adversary_admissible correct adv"

  — Decompose the augmented CD solver’s output.
  obtain F' b L where
    decomp: "cd_alg' (adv_i adv) (adv_e_star adv) = (F', b, L)"
    by (cases "cd_alg' (adv_i adv) (adv_e_star adv)") auto

  from augmented adm decomp
  have valid_F': "valid (adv_E adv) F' (adv_e_star adv)"
    and b_True: "b"
    and L_eq: "L = correct"
    by (auto simp: produces_valid_F_with_L_def Let_def)

  let ?out = "bb_from_cd_with_L cd_alg' procs V
              (adv_e_star adv) (adv_i adv)"

  have F_field: "bb_F ?out = F'"
    by (simp add: bb_from_cd_with_L_def decomp)
  have L_field: "bb_L ?out = correct"
    by (simp add: bb_from_cd_with_L_def decomp L_eq)
  have w_field:
    "bb_w ?out =
      (if ( $\forall p \in correct. \neg V p$ ) then False
        else if ( $\forall p \in correct. V p$ ) then True
        else b)"
    by (simp add: bb_from_cd_with_L_def decomp L_eq)

  — Sub-claim 1: F is valid.
  have claim_valid:
    "valid (adv_E adv) (bb_F ?out) (adv_e_star adv)"
    using valid_F' F_field by simp
```

— Sub-claim 2: `bb_w` matches `w_value`. Three branches following the piecewise definition (Section 4.2): `uniform-false`, `uniform-true`, `mixed`.

```

have claim_w:
  "bb_w ?out =
    w_value correct V (adv_E adv) (bb_F ?out) (adv_e_star adv)"
proof -
  consider
    (uniform_false) "∀p ∈ correct. ¬ V p"
  | (uniform_true)  "¬ (∀p ∈ correct. ¬ V p)" "∀p ∈ correct. V p"
  | (mixed)         "¬ (∀p ∈ correct. ¬ V p)"
    "¬ (∀p ∈ correct. V p)"

  by blast
thus ?thesis
proof cases
  case uniform_false
  have "bb_w ?out = False"
    by (simp add: w_field uniform_false)
  moreover have
    "w_value correct V (adv_E adv) (bb_F ?out) (adv_e_star adv) = False"
    by (simp add: w_value_uniform_false[OF uniform_false])
  ultimately show ?thesis by simp
next
  case uniform_true
  have C_ne: "correct ≠ {}" using uniform_true(2) uniform_true(1) by blast
  have inner_reduce:
    "(if (∀p ∈ correct. V p) then True else b) = True"
    using uniform_true(2) by simp
  have outer_reduce:
    "(if (∀p ∈ correct. ¬ V p) then False
      else if (∀p ∈ correct. V p) then True
      else b)
     = (if (∀p ∈ correct. V p) then True else b)"
    using uniform_true(1) by (rule if_not_P)
  have "bb_w ?out = True"
    using w_field outer_reduce inner_reduce by simp
  moreover have
    "w_value correct V (adv_E adv) (bb_F ?out) (adv_e_star adv) = True"
    by (rule w_value_uniform_true[OF C_ne uniform_true(2)])
  ultimately show ?thesis by simp
next
  case mixed
  have "bb_w ?out = b"
    by (simp add: w_field mixed)
  hence bb_w_True: "bb_w ?out = True" using b_True by simp
  have
    "w_value correct V (adv_E adv) (bb_F ?out) (adv_e_star adv)
      = valid (adv_E adv) (bb_F ?out) (adv_e_star adv)"
    by (rule w_value_mixed[OF mixed(1) mixed(2)])
  also have
    "... = valid (adv_E adv) F' (adv_e_star adv)"
    by (simp add: F_field)

```

```

    also have "... = True" using valid_F' by simp
    finally show ?thesis using bb_w_True by simp
  qed
qed

— Sub-claim 3: bb_L equals the correct set.
have claim_L: "bb_L ?out = correct" by (rule L_field)

show "bb_correct_output procs correct V
      (adv_E adv) (adv_e_star adv) (adv_i adv) ?out"
  using claim_valid claim_w claim_L
  by (simp add: bb_correct_output_def)
qed

theorem blackbox_reduces_to_cd:
  assumes cd: "produces_valid_F correct cd_alg"
  shows "∃bb_alg. solves_BlackBox procs correct bb_alg"
proof -
  from cd_can_identify_correct[OF cd]
  obtain cd_alg' where
    aug: "produces_valid_F_with_L correct cd_alg'"
    by blast
  have "solves_BlackBox procs correct (bb_from_cd_with_L cd_alg')"
    by (rule bb_from_cd_with_L_correct[OF aug])
  thus ?thesis by blast
qed

corollary CD_solvable_imp_BlackBox_solvable:
  assumes "CD_solvable m correct"
  shows "BlackBox_solvable procs correct"
proof -
  obtain cd_alg where "produces_valid_F correct cd_alg"
    using assms by (auto simp: CD_solvable_def)
  thus ?thesis
    using blackbox_reduces_to_cd by (auto simp: BlackBox_solvable_def)
qed

end — context byzantineSystem_with_identification

end

theory Theorems_1_2
  imports CD
begin

```

19 Natural numbers used by an algorithm's output

For a finitely-supported collected history F , the set of natural numbers appearing as either a local sequence number or a message identifier of any event in $\text{events_of } F$ is finite. Hence a fresh natural number, strictly larger than all those that appear, exists.

```

fun nats_of_event :: "'p event  $\Rightarrow$  nat set" where
  "nats_of_event (Internal _ n)      = {n}"
| "nats_of_event (Send _ n _ m)      = {n, m}"
| "nats_of_event (Receive _ n _ m)   = {n, m}"

definition nats_in_F :: "'p history  $\Rightarrow$  nat set" where
  "nats_in_F F  $\equiv \bigcup_{e \in \text{events\_of } F} \text{nats\_of\_event } e$ "

definition fresh_nat :: "'p history  $\Rightarrow$  nat" where
  "fresh_nat F  $\equiv \text{Suc } (\text{Max } (\text{insert } 0 (\text{nats\_in\_F } F)))$ "

lemma finite_nats_of_event [simp]: "finite (nats_of_event e)"
  by (cases e) auto

lemma finite_nats_in_F:
  assumes "finite (events_of F)"
  shows   "finite (nats_in_F F)"
  using assms unfolding nats_in_F_def by simp

lemma nats_in_F_le_Max:
  assumes "finite (events_of F)" "k  $\in$  nats_in_F F"
  shows   "k  $\leq$  Max (insert 0 (nats_in_F F))"
proof -
  have "finite (insert 0 (nats_in_F F))"
    using assms(1) finite_nats_in_F by simp
  thus ?thesis using assms(2) by (intro Max_ge) auto
qed

lemma fresh_nat_above:
  assumes "finite (events_of F)" "k  $\in$  nats_in_F F"
  shows   "k < fresh_nat F"
proof -
  have "k  $\leq$  Max (insert 0 (nats_in_F F))"
    using assms by (rule nats_in_F_le_Max)
  thus ?thesis by (simp add: fresh_nat_def)
qed

lemma fresh_nat_positive [simp]: "fresh_nat F > 0"
  by (simp add: fresh_nat_def)

  Concrete corollaries: any specific event constructed at fresh_nat F as either its sequence number or its message identifier is outside events_of F.

lemma Internal_at_fresh_nat_not_in_F:
  assumes "finite (events_of F)"
  shows   "Internal p (fresh_nat F)  $\notin$  events_of F"
proof
  let ?e = "Internal p (fresh_nat F)"
  assume H: "?e  $\in$  events_of F"
  have inside: "fresh_nat F  $\in$  nats_of_event ?e" by simp
  have "fresh_nat F  $\in$  nats_in_F F"
    using H inside unfolding nats_in_F_def by blast
  with assms fresh_nat_above show False by fastforce

```

qed

```

lemma Send_at_fresh_nat_not_in_F:
  assumes "finite (events_of F)"
  shows   "Send p n q (fresh_nat F)  $\notin$  events_of F"
proof
  let ?e = "Send p n q (fresh_nat F)"
  assume H: "?e  $\in$  events_of F"
  have inside: "fresh_nat F  $\in$  nats_of_event ?e" by simp
  have "fresh_nat F  $\in$  nats_in_F F"
    using H inside unfolding nats_in_F_def by blast
  with assms fresh_nat_above show False by fastforce
qed

```

```

context process_partition
begin

```

20 Theorem 1: false negatives are unavoidable

Paper, Theorem 1 (Section 4.1):

“It is impossible to prevent false negatives in solving the causality determination problem (Definition 5) as specified by $CD(E, F, e_i^*)$ in an asynchronous unicast/multicast/broadcast-based message passing system with one or more Byzantine processes.”

Paper’s proof sketch:

“In the determination of $e_h^x \rightarrow e_i^*$, a false negative may arise when a send-receive event pair (e_f^u, e_g^v) in a causal chain from e_h^x to e_i^* is missing as per F A Byzantine p_g can suppress letting the rest of the system know of the occurrence of e_g^v or swap the order of occurrence of e_g^v and $e_g^{v'}$ in what it lets the rest of the system know about the occurrence of the two local events.”

Our construction. Given an algorithm alg , pick a correct process p_i and a Byzantine process p_b . Take $e_{star} = \text{Internal } p_i \ 2$, and let $F = \text{fst } (alg \ p_i \ e_{star})$ be the algorithm’s collected history on this input. Use a fresh message id $m = \text{fresh_nat } F$ and build the execution E :

- $E \ p_b = [\text{Send } p_b \ 1 \ p_i \ m];$
- $E \ p_i = [\text{Receive } p_i \ 1 \ p_b \ m, \text{Internal } p_i \ 2];$
- $E \ p = []$ elsewhere.

The Send event has a hb-chain to e_{star} in E (message order plus program order at p_i), but it is not in $\text{events_of } F$ because its message id was chosen fresh. Hence $\text{hb_eval } E \ s \ e_{star}$ holds while $\text{hb_eval } F \ s \ e_{star}$ does not – a false negative.

definition $\text{fn_E} :: "'p \Rightarrow 'p \Rightarrow \text{nat} \Rightarrow 'p \text{ history}"$ **where**
 $\text{"fn_E } p_i \ p_b \ m \equiv$

```

      (λp. if p = p_b then [Send p_b 1 p_i m]
           else if p = p_i then [Receive p_i 1 p_b m, Internal p_i 2]
           else [])"

definition fn_adv :: "'p ⇒ 'p ⇒ nat ⇒ 'p adversary" where
  "fn_adv p_i p_b m ≡
    (| adv_E = fn_E p_i p_b m,
      adv_e_star = Internal p_i 2,
      adv_i = p_i |)"

lemma fn_E_at_pb [simp]:
  assumes "p_b ≠ p_i"
  shows "fn_E p_i p_b m p_b = [Send p_b 1 p_i m]"
  using assms by (simp add: fn_E_def)

lemma fn_E_at_pi:
  assumes "p_b ≠ p_i"
  shows "fn_E p_i p_b m p_i = [Receive p_i 1 p_b m, Internal p_i 2]"
  using assms by (auto simp: fn_E_def)

lemma fn_E_elsewhere:
  assumes "p ≠ p_b" "p ≠ p_i"
  shows "fn_E p_i p_b m p = []"
  using assms by (simp add: fn_E_def)

lemma fn_E_events:
  assumes "p_b ≠ p_i"
  shows "events_of (fn_E p_i p_b m) =
    {Send p_b 1 p_i m, Receive p_i 1 p_b m, Internal p_i 2}"
proof -
  have "events_of (fn_E p_i p_b m) = (⋃p. set (fn_E p_i p_b m p))"
    by (simp add: events_of_def)
  also have "... = set (fn_E p_i p_b m p_b) ∪ set (fn_E p_i p_b m p_i)
    ∪ (⋃p ∈ -{p_b, p_i}. set (fn_E p_i p_b m p))"
    by auto
  also have "set (fn_E p_i p_b m p_b) = {Send p_b 1 p_i m}"
    using assms by simp
  moreover have "set (fn_E p_i p_b m p_i)
    = {Receive p_i 1 p_b m, Internal p_i 2}"
    using assms by (simp add: fn_E_at_pi)
  moreover have "(⋃p ∈ -{p_b, p_i}. set (fn_E p_i p_b m p)) = {}"
    by (auto simp: fn_E_elsewhere)
  ultimately show ?thesis by auto
qed

lemma wf_history_local_pb_in_fn_E:
  assumes "p_b ≠ p_i"
  shows "wf_history_local p_b (fn_E p_i p_b m p_b)"
proof -
  have list_eq: "fn_E p_i p_b m p_b = [Send p_b 1 p_i m]"
    using assms by simp
  have "∀e ∈ set [Send p_b 1 p_i m]. proc_of e = p_b" by simp

```

```

    moreover have "∀k < length [Send p_b 1 p_i m].
      seq_of ([Send p_b 1 p_i m] ! k) = Suc k"
    by (simp add: less_Suc_eq)
  ultimately show ?thesis
    using list_eq unfolding wf_history_local_def by simp
qed

lemma wf_history_local_pi_in_fn_E:
  assumes "p_b ≠ p_i"
  shows "wf_history_local p_i (fn_E p_i p_b m p_i)"
proof -
  let ?l = "[Receive p_i 1 p_b m, Internal p_i 2]"
  have list_eq: "fn_E p_i p_b m p_i = ?l"
    using assms by (rule fn_E_at_pi)
  have proc_ok: "∀e ∈ set ?l. proc_of e = p_i" by simp
  have seq_ok: "∀k < length ?l. seq_of (?l ! k) = Suc k"
  proof (intro allI impI)
    fix k assume "k < length ?l"
    hence "k = 0 ∨ k = 1" by auto
    thus "seq_of (?l ! k) = Suc k" by auto
  qed
  show ?thesis using list_eq proc_ok seq_ok
    unfolding wf_history_local_def by simp
qed

lemma wf_history_local_elsewhere_in_fn_E:
  assumes "p ≠ p_b" "p ≠ p_i"
  shows "wf_history_local p (fn_E p_i p_b m p)"
  using assms by (simp add: fn_E_elsewhere wf_history_local_def)

lemma wf_history_fn_E:
  assumes "p_b ≠ p_i"
  shows "wf_history (fn_E p_i p_b m)"
proof (unfold wf_history_def, intro allI)
  fix p
  show "wf_history_local p (fn_E p_i p_b m p)"
  proof (cases "p = p_b")
    case True
    thus ?thesis using assms wf_history_local_pb_in_fn_E by simp
  next
    case False
    show ?thesis
    proof (cases "p = p_i")
      case True
      thus ?thesis using assms wf_history_local_pi_in_fn_E by simp
    next
      case False
      with <p ≠ p_b> show ?thesis
        by (rule wf_history_local_elsewhere_in_fn_E)
    qed
  qed
qed

```

```

lemma program_order_fn_E_at_pi:
  assumes "p_b ≠ p_i"
  shows "program_order (fn_E p_i p_b m)
        (Receive p_i 1 p_b m) (Internal p_i 2)"
proof -
  let ?H = "fn_E p_i p_b m"
  have list_eq: "?H p_i = [Receive p_i 1 p_b m, Internal p_i 2]"
    using assms by (rule fn_E_at_pi)
  have len: "length (?H p_i) = 2" by (simp add: list_eq)
  have e0: "(?H p_i) ! 0 = Receive p_i 1 p_b m" by (simp add: list_eq)
  have e1: "(?H p_i) ! 1 = Internal p_i 2" by (simp add: list_eq)
  have "(0::nat) < 1" by simp
  moreover have "(1::nat) < length (?H p_i)" using len by simp
  ultimately show ?thesis
    using e0 e1 unfolding program_order_def by blast
qed

lemma message_order_fn_E_send_receive:
  assumes "p_b ≠ p_i"
  shows "message_order (fn_E p_i p_b m)
        (Send p_b 1 p_i m) (Receive p_i 1 p_b m)"
  using assms unfolding message_order_def
  by (simp add: fn_E_events)

lemma hb_send_to_estar_in_fn_E:
  assumes "p_b ≠ p_i"
  shows "hb (fn_E p_i p_b m) (Send p_b 1 p_i m) (Internal p_i 2)"
proof -
  let ?H = "fn_E p_i p_b m"
  let ?s = "Send p_b 1 p_i m"
  let ?r = "Receive p_i 1 p_b m"
  let ?es = "Internal p_i 2"
  have step1: "hb_step ?H ?s ?r"
    using message_order_fn_E_send_receive[OF assms]
    by (simp add: hb_step_def)
  have step2: "hb_step ?H ?r ?es"
    using program_order_fn_E_at_pi[OF assms] by (simp add: hb_step_def)
  have base: "(hb_step ?H)++ ?s ?r"
    using step1 by blast
  have extend: "(hb_step ?H)++ ?r ?es"
    using step2 by blast
  have "(hb_step ?H)++ ?s ?es"
    using base extend by (rule tranclp_trans)
  thus ?thesis by (simp add: hb_def)
qed

lemma hb_eval_send_to_estar_in_fn_E:
  assumes "p_b ≠ p_i"
  shows "hb_eval (fn_E p_i p_b m) (Send p_b 1 p_i m) (Internal p_i 2)"
proof -
  have e1: "Send p_b 1 p_i m ∈ events_of (fn_E p_i p_b m)"

```

```

    using assms by (simp add: fn_E_events)
  have e2: "Internal p_i 2 ∈ events_of (fn_E p_i p_b m)"
    using assms by (simp add: fn_E_events)
  have h: "hb (fn_E p_i p_b m) (Send p_b 1 p_i m) (Internal p_i 2)"
    using assms by (rule hb_send_to_estar_in_fn_E)
  show ?thesis using e1 e2 h by (simp add: hb_eval_def)
qed

lemma adv_admissible_fn_adv:
  assumes "p_i ∈ correct" "p_b ≠ p_i"
  shows "adversary_admissible correct (fn_adv p_i p_b m)"
proof -
  have a: "wf_history (fn_E p_i p_b m)"
    using assms(2) by (rule wf_history_fn_E)
  have b: "p_i ∈ correct" by (rule assms(1))
  have c: "proc_of (Internal p_i 2) = p_i" by simp
  have d: "Internal p_i 2 ∈ events_of (fn_E p_i p_b m)"
    using assms(2) by (simp add: fn_E_events)
  show ?thesis using a b c d
    by (simp add: adversary_admissible_def fn_adv_def)
qed

theorem CD_FN_unavoidable:
  assumes byz_cor_distinct:
    "∃p_i p_b. p_i ∈ correct ∧ p_b ∈ byzantine ∧ p_b ≠ p_i"
  and fin_F: "∀p_i_in. finite (events_of
    (fst (alg p_i_in (Internal p_i_in 2))))"
  shows "∃adv. adversary_admissible correct adv ∧
    false_negative (adv_E adv)
    (fst (alg (adv_i adv) (adv_e_star adv)))
    (adv_e_star adv)"
proof -
  obtain p_i p_b where
    pi_cor: "p_i ∈ correct" and pb_byz: "p_b ∈ byzantine"
    and dist: "p_b ≠ p_i"
    using byz_cor_distinct by blast

  let ?e_star = "Internal p_i 2"
  let ?F = "fst (alg p_i ?e_star)"
  let ?m = "fresh_nat ?F"
  let ?adv = "fn_adv p_i p_b ?m"
  let ?s = "Send p_b 1 p_i ?m"

  have finF: "finite (events_of ?F)" using fin_F by blast

  have adm: "adversary_admissible correct ?adv"
    using pi_cor dist by (rule adv_admissible_fn_adv)

  have witness_in_E: "?s ∈ events_of (adv_E ?adv)"
    using dist by (simp add: fn_adv_def fn_E_events)
  have hbE: "hb_eval (adv_E ?adv) ?s (adv_e_star ?adv)"
proof -

```

```

    have "hb_eval (fn_E p_i p_b ?m) ?s (Internal p_i 2)"
      by (rule hb_eval_send_to_estar_in_fn_E[OF dist])
    thus ?thesis by (simp add: fn_adv_def)
  qed
  have witness_not_in_F: "?s ∉ events_of ?F"
    using finF Send_at_fresh_nat_not_in_F[of ?F] by blast
  have not_hbF: "¬ hb_eval ?F ?s (adv_e_star ?adv)"
    using witness_not_in_F by (simp add: hb_eval_def fn_adv_def)

  from witness_in_E hbE not_hbF
  have "∃ e ∈ events_of (adv_E ?adv) ∪ events_of ?F.
    hb_eval (adv_E ?adv) e (adv_e_star ?adv) ∧
    ¬ hb_eval ?F e (adv_e_star ?adv)"
    by blast
  hence FN: "false_negative (adv_E ?adv) ?F (adv_e_star ?adv)"
    by (simp add: false_negative_def)

  have F_eq: "fst (alg (adv_i ?adv) (adv_e_star ?adv)) = ?F"
    by (simp add: fn_adv_def)

  from adm FN F_eq show ?thesis by metis
  qed

```

21 Theorem 2: false negatives or false positives are unavoidable for internal events

Paper, Theorem 2 (Section 4.1):

“For an internal event e_h^x , it is impossible to prevent false negatives or false positives in determining $e_h^x \rightarrow e_i^*$ at a correct process p_i in an asynchronous message passing system with one or more Byzantine processes.”

Paper’s proof sketch:

“There may be no other event in the rest of the system to corroborate the occurrence of an internal event at a process. A Byzantine process p_h can choose to not reveal about an internal event e_h^x to the rest of the system, leading to a false negative that cannot be prevented. It may also choose to add a fake internal event e_h^x in what it reveals to the rest of the system, leading to a false positive that cannot be prevented.”

Our construction. We strengthen the Theorem 1 construction so that the witness event is an Internal event at a Byzantine process, rather than a Send event. The Byzantine process p_b performs a chain of k internal events followed by a single Send to the correct target p_i , where $k = \text{fresh_nat } F$ is chosen so that the k -th internal event has a sequence number absent from $\text{events_of } F$. The internal event still has an hb-chain to e_star in E (through subsequent program order to the Send, then message order, then program order at p_i); but the internal event itself is not in $\text{events_of } F$, so a false negative arises.

Deviation: the paper’s Theorem 2 is an FN-or-FP disjunction (“the Byzantine p_h can choose to not reveal ... OR ... add a fake internal event”). We discharge the FN side

constructively. Since the statement is a disjunction ($FN \vee FP$), exhibiting an adversary that produces FN is sufficient.

definition `fn_internal_E` :: "'p $\Rightarrow$ 'p $\Rightarrow$ nat $\Rightarrow$ nat $\Rightarrow$ 'p history" **where**

```
"fn_internal_E p_i p_b k m  $\equiv$ 
  ( $\lambda$ p. if p = p_b
    then (map (Internal p_b) [1.. $\text{Suc } k$ ])
      @ [Send p_b (Suc k) p_i m]
    else if p = p_i
      then [Receive p_i 1 p_b m, Internal p_i 2]
    else [])"
```

definition `fn_internal_adv` :: "'p $\Rightarrow$ 'p $\Rightarrow$ nat $\Rightarrow$ nat $\Rightarrow$ 'p adversary" **where**

```
"fn_internal_adv p_i p_b k m  $\equiv$ 
  ( $\lambda$  adv_E = fn_internal_E p_i p_b k m,
    adv_e_star = Internal p_i 2,
    adv_i = p_i  $\lambda$ )"
```

lemma `fn_internal_E_at_pb` [simp]:

```
assumes "p_b  $\neq$  p_i"
shows "fn_internal_E p_i p_b k m p_b
  = (map (Internal p_b) [1.. $\text{Suc } k$ ]) @ [Send p_b (Suc k) p_i m]"
using assms by (simp add: fn_internal_E_def)
```

lemma `fn_internal_E_at_pi`:

```
assumes "p_b  $\neq$  p_i"
shows "fn_internal_E p_i p_b k m p_i
  = [Receive p_i 1 p_b m, Internal p_i 2]"
using assms by (auto simp: fn_internal_E_def)
```

lemma `fn_internal_E_elsewhere`:

```
assumes "p  $\neq$  p_b" "p  $\neq$  p_i"
shows "fn_internal_E p_i p_b k m p = []"
using assms by (simp add: fn_internal_E_def)
```

lemma `length_fn_internal_E_pb`:

```
assumes "p_b  $\neq$  p_i"
shows "length (fn_internal_E p_i p_b k m p_b) = Suc k"
using assms by simp
```

lemma `nth_fn_internal_E_pb_lt`:

```
assumes "p_b  $\neq$  p_i" "j < k"
shows "fn_internal_E p_i p_b k m p_b ! j = Internal p_b (Suc j)"
```

proof -

```
let ?xs = "map (Internal p_b) [1.. $\text{Suc } k$ ]"
let ?L = "?xs @ [Send p_b (Suc k) p_i m]"
have list_eq: "fn_internal_E p_i p_b k m p_b = ?L"
  using assms(1) by simp
have len_xs: "length ?xs = k"
  using assms(2) by simp
with assms(2) have j_lt_len: "j < length ?xs" by simp
have j_lt_upt: "j < length [1.. $\text{Suc } k$ ]"
  using assms(2) by simp
```

```

have step1: "?L ! j = ?xs ! j"
  using j_lt_len by (simp add: nth_append)
have step2: "?xs ! j = Internal p_b ([1..<Suc k] ! j)"
  using j_lt_upt by (rule nth_map)
have step3: "[1..<Suc k] ! j = Suc j"
proof -
  have "[1..<Suc k] ! j = 1 + j"
    using assms(2) by (intro nth_upt) simp
  thus ?thesis by simp
qed
from step1 step2 step3 list_eq show ?thesis by simp
qed

lemma nth_fn_internal_E_pb_eq:
  assumes "p_b ≠ p_i"
  shows "fn_internal_E p_i p_b k m p_b ! k = Send p_b (Suc k) p_i m"
  using assms by (simp add: nth_append)

lemma set_fn_internal_E_pb:
  assumes "p_b ≠ p_i" "k ≥ 1"
  shows "set (fn_internal_E p_i p_b k m p_b)
    = (Internal p_b ' {1..k}) ∪ {Send p_b (Suc k) p_i m}"
proof -
  have set_map: "set (map (Internal p_b) [1..<Suc k]) = Internal p_b ' {1..<Suc k}"
    by auto
  have rng_eq: "{1..<Suc k} = {1..k}" by auto
  have "set (fn_internal_E p_i p_b k m p_b)
    = set (map (Internal p_b) [1..<Suc k]) ∪ {Send p_b (Suc k) p_i m}"
    using assms(1) by simp
  also have "... = (Internal p_b ' {1..k}) ∪ {Send p_b (Suc k) p_i m}"
    using set_map rng_eq by simp
  finally show ?thesis .
qed

lemma fn_internal_E_events:
  assumes "p_b ≠ p_i" "k ≥ 1"
  shows "events_of (fn_internal_E p_i p_b k m) =
    (Internal p_b ' {1..k})
    ∪ {Send p_b (Suc k) p_i m,
      Receive p_i 1 p_b m,
      Internal p_i 2}"
proof -
  have "events_of (fn_internal_E p_i p_b k m)
    = (⋃p. set (fn_internal_E p_i p_b k m p))"
    by (simp add: events_of_def)
  also have "... = set (fn_internal_E p_i p_b k m p_b)
    ∪ set (fn_internal_E p_i p_b k m p_i)
    ∪ (⋃p ∈ -{p_b, p_i}. set (fn_internal_E p_i p_b k m p))"
    by auto
  also have "(⋃p ∈ -{p_b, p_i}. set (fn_internal_E p_i p_b k m p)) = {}"
    by (auto simp: fn_internal_E_elsewhere)
  moreover have "set (fn_internal_E p_i p_b k m p_b)

```

```

      = (Internal p_b ' {1..k}) ∪ {Send p_b (Suc k) p_i m}"
    using assms by (rule set_fn_internal_E_pb)
  moreover have "set (fn_internal_E p_i p_b k m p_i)
    = {Receive p_i 1 p_b m, Internal p_i 2}"
    using assms(1) by (simp add: fn_internal_E_at_pi)
  ultimately show ?thesis by auto
qed

```

```

lemma wf_history_local_pb_in_fn_internal_E:
  assumes "p_b ≠ p_i" "k ≥ 1"
  shows "wf_history_local p_b (fn_internal_E p_i p_b k m p_b)"
proof -
  let ?L = "fn_internal_E p_i p_b k m p_b"
  have len_L: "length ?L = Suc k" using assms(1) by simp

  have all_proc: "∀e ∈ set ?L. proc_of e = p_b"
  proof
    fix e assume "e ∈ set ?L"
    hence "e ∈ (Internal p_b ' {1..k}) ∪ {Send p_b (Suc k) p_i m}"
      using set_fn_internal_E_pb[OF assms] by simp
    thus "proc_of e = p_b" by (auto simp: image_iff)
  qed

  have all_seq: "∀j < length ?L. seq_of (?L ! j) = Suc j"
  proof (intro allI impI)
    fix j assume "j < length ?L"
    hence j_lt: "j < Suc k" using len_L by simp
    show "seq_of (?L ! j) = Suc j"
    proof (cases "j < k")
      case True
      thus ?thesis using nth_fn_internal_E_pb_lt[OF assms(1)] by simp
    next
      case False
      with j_lt have "j = k" by simp
      thus ?thesis using nth_fn_internal_E_pb_eq[OF assms(1)] by simp
    qed
  qed

  show ?thesis using all_proc all_seq
    unfolding wf_history_local_def by blast
qed

```

```

lemma wf_history_local_pi_in_fn_internal_E:
  assumes "p_b ≠ p_i"
  shows "wf_history_local p_i (fn_internal_E p_i p_b k m p_i)"
proof -
  let ?L = "[Receive p_i 1 p_b m, Internal p_i 2]"
  have list_eq: "fn_internal_E p_i p_b k m p_i = ?L"
    using assms by (rule fn_internal_E_at_pi)
  have proc_ok: "∀e ∈ set ?L. proc_of e = p_i" by simp
  have seq_ok: "∀j < length ?L. seq_of (?L ! j) = Suc j"
  proof (intro allI impI)

```

```

    fix j assume "j < length ?L"
    hence "j = 0  $\vee$  j = 1" by auto
    thus "seq_of (?L ! j) = Suc j" by auto
qed
show ?thesis using list_eq proc_ok seq_ok
  unfolding wf_history_local_def by simp
qed

lemma wf_history_local_elsewhere_in_fn_internal_E:
  assumes "p  $\neq$  p_b" "p  $\neq$  p_i"
  shows "wf_history_local p (fn_internal_E p_i p_b k m p)"
  using assms by (simp add: fn_internal_E_elsewhere wf_history_local_def)

lemma wf_history_fn_internal_E:
  assumes "p_b  $\neq$  p_i" "k  $\geq$  1"
  shows "wf_history (fn_internal_E p_i p_b k m)"
proof (unfold wf_history_def, intro allI)
  fix p
  show "wf_history_local p (fn_internal_E p_i p_b k m p)"
  proof (cases "p = p_b")
    case True
    thus ?thesis using assms wf_history_local_pb_in_fn_internal_E by simp
  next
    case False
    show ?thesis
    proof (cases "p = p_i")
      case True
      thus ?thesis using assms(1) wf_history_local_pi_in_fn_internal_E by simp
    next
      case False
      with <p  $\neq$  p_b> show ?thesis
      by (rule wf_history_local_elsewhere_in_fn_internal_E)
    qed
  qed
qed
qed

lemma program_order_internal_to_send_at_pb:
  assumes "p_b  $\neq$  p_i" "k  $\geq$  1"
  shows "program_order (fn_internal_E p_i p_b k m)
    (Internal p_b k) (Send p_b (Suc k) p_i m)"
proof -
  let ?H = "fn_internal_E p_i p_b k m"
  have len: "length (?H p_b) = Suc k" using assms(1) by simp
  from assms(2) have km1_lt_k: "k - 1 < k" by simp
  have at_km1: "(?H p_b) ! (k - 1) = Internal p_b (Suc (k - 1))"
    using assms(1) km1_lt_k nth_fn_internal_E_pb_lt by simp
  hence at_km1': "(?H p_b) ! (k - 1) = Internal p_b k"
    using assms(2) by simp
  have at_k: "(?H p_b) ! k = Send p_b (Suc k) p_i m"
    using assms(1) by (rule nth_fn_internal_E_pb_eq)
  have idx1: "k - 1 < k" using assms(2) by simp
  have idx2: "k < length (?H p_b)" using len by simp

```

```

    show ?thesis using at_km1' at_k idx1 idx2
      unfolding program_order_def by blast
qed

lemma program_order_receive_to_estar_in_fn_internal_E:
  assumes "p_b ≠ p_i"
  shows "program_order (fn_internal_E p_i p_b k m)
    (Receive p_i 1 p_b m) (Internal p_i 2)"
proof -
  let ?H = "fn_internal_E p_i p_b k m"
  have list_eq: "?H p_i = [Receive p_i 1 p_b m, Internal p_i 2]"
    using assms by (rule fn_internal_E_at_pi)
  have len: "length (?H p_i) = 2" by (simp add: list_eq)
  have e0: "(?H p_i) ! 0 = Receive p_i 1 p_b m" by (simp add: list_eq)
  have e1: "(?H p_i) ! 1 = Internal p_i 2" by (simp add: list_eq)
  have "(0::nat) < 1" by simp
  moreover have "(1::nat) < length (?H p_i)" using len by simp
  ultimately show ?thesis using e0 e1
    unfolding program_order_def by blast
qed

lemma message_order_in_fn_internal_E:
  assumes "p_b ≠ p_i" "k ≥ 1"
  shows "message_order (fn_internal_E p_i p_b k m)
    (Send p_b (Suc k) p_i m) (Receive p_i 1 p_b m)"
  using assms unfolding message_order_def
  by (simp add: fn_internal_E_events)

lemma hb_internal_to_estar_in_fn_internal_E:
  assumes "p_b ≠ p_i" "k ≥ 1"
  shows "hb (fn_internal_E p_i p_b k m) (Internal p_b k) (Internal p_i 2)"
proof -
  let ?H = "fn_internal_E p_i p_b k m"
  let ?ih = "Internal p_b k"
  let ?s = "Send p_b (Suc k) p_i m"
  let ?r = "Receive p_i 1 p_b m"
  let ?es = "Internal p_i 2"
  have step1: "hb_step ?H ?ih ?s"
    using program_order_internal_to_send_at_pb[OF assms]
    by (simp add: hb_step_def)
  have step2: "hb_step ?H ?s ?r"
    using message_order_in_fn_internal_E[OF assms]
    by (simp add: hb_step_def)
  have step3: "hb_step ?H ?r ?es"
    using program_order_receive_to_estar_in_fn_internal_E[OF assms(1)]
    by (simp add: hb_step_def)
  have t1: "(hb_step ?H)++ ?ih ?s" using step1 by blast
  have t2: "(hb_step ?H)++ ?s ?r" using step2 by blast
  have t3: "(hb_step ?H)++ ?r ?es" using step3 by blast
  have "(hb_step ?H)++ ?ih ?r" using t1 t2 by (rule tranclp_trans)
  hence "(hb_step ?H)++ ?ih ?es" using t3 by (rule tranclp_trans)
  thus ?thesis by (simp add: hb_def)

```

qed

lemma hb_eval_internal_to_estar_in_fn_internal_E:

assumes "p_b $\neq$ p_i" "k $\geq$ 1"
 shows "hb_eval (fn_internal_E p_i p_b k m)
 (Internal p_b k) (Internal p_i 2)"

proof -

let ?H = "fn_internal_E p_i p_b k m"
 have e_ih: "Internal p_b k $\in$ events_of ?H"
 using assms by (simp add: fn_internal_E_events)
 have e_es: "Internal p_i 2 $\in$ events_of ?H"
 using assms by (simp add: fn_internal_E_events)
 have h: "hb ?H (Internal p_b k) (Internal p_i 2)"
 using assms by (rule hb_internal_to_estar_in_fn_internal_E)
 show ?thesis using e_ih e_es h by (simp add: hb_eval_def)

qed

lemma adv_admissible_fn_internal_adv:

assumes "p_i $\in$ correct" "p_b $\neq$ p_i" "k $\geq$ 1"
 shows "adversary_admissible correct (fn_internal_adv p_i p_b k m)"

proof -

have a: "wf_history (fn_internal_E p_i p_b k m)"
 using assms(2) assms(3) by (rule wf_history_fn_internal_E)
 have b: "p_i $\in$ correct" by (rule assms(1))
 have c: "proc_of (Internal p_i 2) = p_i" by simp
 have d: "Internal p_i 2 $\in$ events_of (fn_internal_E p_i p_b k m)"
 using assms(2) assms(3) by (simp add: fn_internal_E_events)
 show ?thesis using a b c d
 by (simp add: adversary_admissible_def fn_internal_adv_def)

qed

theorem CD_FN_or_FP_unavoidable_internal:

assumes byz_cor_distinct:
 $\exists p_i p_b. p_i \in \text{correct} \wedge p_b \in \text{byzantine} \wedge p_b \neq p_i$
 and fin_F: " $\forall p_{i_in}. \text{finite} (\text{events_of}$
 $(\text{fst} (\text{alg } p_{i_in} (\text{Internal } p_{i_in} 2))))$ "
 shows " $\exists \text{adv } e_h. \text{adversary_admissible correct adv} \wedge$
 $(\exists p_n. e_h = \text{Internal } p_n) \wedge$
 $((\text{hb_eval } (\text{adv_E adv}) e_h (\text{adv_e_star adv}) \wedge$
 $\neg \text{hb_eval } (\text{fst } (\text{alg } (\text{adv_i adv}) (\text{adv_e_star adv})))$
 $e_h (\text{adv_e_star adv}))$
 $\vee$
 $(\neg \text{hb_eval } (\text{adv_E adv}) e_h (\text{adv_e_star adv}) \wedge$
 $\text{hb_eval } (\text{fst } (\text{alg } (\text{adv_i adv}) (\text{adv_e_star adv})))$
 $e_h (\text{adv_e_star adv})))$ "

proof -

obtain p_i p_b where
 pi_cor: "p_i $\in$ correct" and pb_byz: "p_b $\in$ byzantine"
 and dist: "p_b $\neq$ p_i"
 using byz_cor_distinct by blast

let ?e_star = "Internal p_i 2"

```

let ?F = "fst (alg p_i ?e_star)"
let ?k = "fresh_nat ?F"
let ?m = "fresh_nat ?F"
let ?adv = "fn_internal_adv p_i p_b ?k ?m"
let ?ih = "Internal p_b ?k"

have finF: "finite (events_of ?F)" using fin_F by blast

have k_pos: "?k ≥ 1"
  by (simp add: fresh_nat_def Suc_leI)

have adm: "adversary_admissible correct ?adv"
  using pi_cor dist k_pos by (rule adv_admissible_fn_internal_adv)

have witness_in_E: "?ih ∈ events_of (adv_E ?adv)"
  using dist k_pos
  by (auto simp: fn_internal_adv_def fn_internal_E_events)
have hbE: "hb_eval (adv_E ?adv) ?ih (adv_e_star ?adv)"
proof -
  have "hb_eval (fn_internal_E p_i p_b ?k ?m) ?ih (Internal p_i 2)"
    by (rule hb_eval_internal_to_estar_in_fn_internal_E[OF dist k_pos])
  thus ?thesis by (simp add: fn_internal_adv_def)
qed
have witness_not_in_F: "?ih ∉ events_of ?F"
  using finF Internal_at_fresh_nat_not_in_F[of ?F] by blast
have not_hbF: "¬ hb_eval ?F ?ih (adv_e_star ?adv)"
  using witness_not_in_F by (simp add: hb_eval_def fn_internal_adv_def)

have F_eq: "fst (alg (adv_i ?adv) (adv_e_star ?adv)) = ?F"
  by (simp add: fn_internal_adv_def)

have e_h_internal: "∃p n. ?ih = Internal p n" by blast

from adm e_h_internal hbE not_hbF F_eq witness_in_E
show ?thesis by metis
qed

end — context process_partition

end

theory FLP_Consensus
  imports
    BlackBox
    FLP.AsynchronousSystem
    FLP.Execution
    FLP.FLPSystem
    FLP.FLPTheorem
begin

```

22 FLP-style consensus solvability

Paper, Section 4.2 (the Consensus problem):

“In the Consensus problem, each process has an initial value and all correct processes must agree on a single value. The solution needs to satisfy the following three conditions.

- **Agreement:** All non-faulty processes must agree on the same single value.
- **Validity:** If all non-faulty processes have the same initial value, then the agreed-on value by all the non-faulty processes must be that same value.
- **Termination:** Each non-faulty process must eventually decide on a value.

According to the FLP impossibility result, it is impossible to solve Consensus in an asynchronous message-passing system with even a single crash failure prone process.”

An algorithm in the FLP model (as formalised by the AFP entry) is a triple (`transFn`, `sendsFn`, `startFn`) of a transition function, a send function, and an initial-state function. We say the triple `flp-solves consensus` when it satisfies the AFP entry’s `flpSystem` and `flpPseudoConsensus` locale axioms together with the three correctness conditions that the AFP entry’s `flpPseudoConsensus.ConsensusFails` takes as preconditions:

- `flpSystem.terminationFLP`: every fair infinite execution terminates (paper’s Termination, formalised in AFP);
- `flpSystem.validity`: validity (paper’s Validity);
- `flpSystem.agreementInit`: agreement on initial configurations (paper’s Agreement).

definition `flp_consensus_solvable ::`

```
"('p ⇒ 's ⇒ 'v messageValue ⇒ 's)
⇒ ('p ⇒ 's ⇒ 'v messageValue ⇒ ('p, 'v) message multiset)
⇒ ('p ⇒ 's)
⇒ bool" where
"flp_consensus_solvable transFn sendsFn startFn <=>
  flpSystem sendsFn ∧
  flpPseudoConsensus transFn sendsFn startFn ∧
  (∀ fe ft. asynchronousSystem.fairInfiniteExecution transFn sendsFn startFn fe
ft
  → flpSystem.terminationFLP transFn sendsFn startFn fe ft) ∧
  (∀ i c. flpSystem.validity transFn sendsFn startFn i c) ∧
  (∀ i c. flpSystem.agreementInit transFn sendsFn startFn i c)"
```

The FLP impossibility, on this predicate, is a one-step invocation of the AFP entry’s `flpPseudoConsensus.ConsensusFails` theorem (proven, not assumed):

“theorem `ConsensusFails`: assumes Termination: ... and Validity: ... and Agreement: ... shows False”

This is the genuine FLP result. We unpack the predicate into its five conjuncts, interpret `flpPseudoConsensus`, and apply `ConsensusFails`.

```

theorem flp_consensus_unsolvable:
  shows "¬ flp_consensus_solvable transFn sendsFn startFn"
proof
  assume A: "flp_consensus_solvable transFn sendsFn startFn"
  hence sys: "flpSystem sendsFn"
    and pc: "flpPseudoConsensus transFn sendsFn startFn"
    and termFLP:
      "\fe ft. asynchronousSystem.fairInfiniteExecution
        transFn sendsFn startFn fe ft  $\implies$ 
        flpSystem.terminationFLP transFn sendsFn startFn fe ft"
  and val: "\i c. flpSystem.validity transFn sendsFn startFn i c"
  and agr: "\i c. flpSystem.agreementInit transFn sendsFn startFn i c"
  by (auto simp: flp_consensus_solvable_def)

interpret flpPseudoConsensus transFn sendsFn startFn using pc .

have "False"
  using flpPseudoConsensus.ConsensusFails[OF pc termFLP val agr] .
thus False .
qed

```

23 BlackBox-to-FLP bridge predicate

Paper, Section 4.2 (the reduction $\text{Consensus} \preceq \text{Black_Box}$):

“Now we give the reduction from Consensus to Black_Box. To solve Consensus(v) at (a correct process) p_i , we invoke Black_Box(v, E, F, e_i^*) locally Each correct process computes $\min(L)$ from the locally returned list L and outputs as its consensus value the broadcast value that it receives from $p_{\min}(L)$ and terminates. The conditions of Consensus – Agreement, Validity, and Termination – can be seen to be satisfied. So Consensus $\preceq$ Black_Box.”

Deviation – meta-level step. The paper’s reduction is operational (“invoke BB locally, broadcast, collect, decide”). We cannot mechanise this at the abstract level of `solves_BlackBox` because that predicate has no notion of an asynchronous distributed execution, fairness, or termination under failure. We therefore capture the reduction as the named predicate below:

“If an abstract BlackBox solver for (P, C) exists, then – in the AFP entry’s FLP model – some triple $(\text{transFn}, \text{sendsFn}, \text{startFn})$ of state-transition functions flp-solves consensus.”

This is the right semantic content of the paper’s reduction. The predicate takes two type witnesses ($'s$ for state, $'v$ for message-value type) because HOL cannot existentially quantify over types. A user discharging this predicate would exhibit specific $'s, 'v, \text{transFn}, \text{sendsFn}, \text{startFn}$ implementing the “broadcast V , collect, invoke BB, decide” protocol of the paper.

Faithfulness: the bridge is a hypothesis (not an axiom) that the impossibility theorems (3, 4, 5 in `Impossibility.thy`) take explicitly. It replaces the formerly-vacuous `byzantineSystem.flp_consensus_impossibility` axiom; see `Foundation_Vacuity.thy` for the historical issue.

```

definition bb_realizes_flp_consensus ::
  "'p set  $\Rightarrow$  'p set  $\Rightarrow$  's itself  $\Rightarrow$  'v itself  $\Rightarrow$  bool" where
  "bb_realizes_flp_consensus P C ts tv  $\longleftrightarrow$ 
    BlackBox_solvable P C  $\longrightarrow$ 
      ( $\exists$  (transFn :: 'p  $\Rightarrow$  's  $\Rightarrow$  'v messageValue  $\Rightarrow$  's)
        (sendsFn :: 'p  $\Rightarrow$  's  $\Rightarrow$  'v messageValue  $\Rightarrow$  ('p, 'v) message multiset)
        (startFn :: 'p  $\Rightarrow$  's).
        flp_consensus_solvable transFn sendsFn startFn)"

```

Composing the bridge with $\neg$ `flp_consensus_solvable ?transFn ?sendsFn ?startFn` yields the form actually used in `Impossibility.thy`:

```

lemma BlackBox_unsolvable_via_bridge:
  assumes bridge: "bb_realizes_flp_consensus P C TYPE('s) TYPE('v)"
  shows " $\neg$  BlackBox_solvable P C"
proof
  assume BB: "BlackBox_solvable P C"
  from bridge[unfolded bb_realizes_flp_consensus_def] BB
  show False
  using flp_consensus_unsolvable by blast
qed

end

```

```

theory Impossibility
  imports Reductions FLP_Consensus
begin

```

24 Theorem 3: CD impossible under unicast

Paper, Theorem 3 (Section 4.2):

“It is impossible to solve causality determination (Definition 5) as specified by $\text{CD}(\mathbf{E}, \mathbf{F}, \mathbf{e}_i^*)$ in an asynchronous unicast-based message passing system with one or more Byzantine processes.”

The paper’s proof composes the two reductions of Section 4.2 with FLP’s impossibility:

“Transitivity of reductions implies that if the CD problem is solvable, then Consensus is also solvable. However, that contradicts the FLP impossibility result [35] when applied to a Byzantine system, hence CD cannot be solvable.”

Deviation – non-vacuous chain. At our abstraction level the pure-HOL `solves_Consensus` predicate is too weak to express FLP (`Foundation_Vacuity.thy` makes this machine-checked). We therefore route the chain through the FLP-style `flp_consensus_solvable`

predicate of `FLP_Consensus.thy` instead, using the meta-level bridge `bb_realizes_flp_consensus` to cross from BB-solvability to “some asynchronous distributed protocol flp-solves consensus”. The chain is then:

$$\begin{aligned} & \text{CD_solvable} \longrightarrow \text{BlackBox_solvable} \text{ (by R2 + cd_can_identify_correct)} \longrightarrow \\ & \exists \text{ FLP-style consensus solver (by the bridge)} \longrightarrow \text{False (by flp_consensus_unsolvable,} \\ & \text{proven against the AFP entry's ConsensusFails).} \end{aligned}$$

The two type witnesses `'s` (state) and `'v` (message-value) are parameters of the bridge predicate.

```
context byzantineSystem_with_identification
begin
```

```
theorem CD_impossible_unicast:
  assumes byz_ne: "byzantine ≠ {}"
    and cor_ne: "correct ≠ {}"
    and bridge: "bb_realizes_flp_consensus
                  procs correct TYPE('s) TYPE('v)"
  shows "¬ CD_solvable Unicast correct"
proof
  assume CD_solv: "CD_solvable Unicast correct"

  — Step 1: CD solvable implies BlackBox solvable.
  have BB_solv: "BlackBox_solvable procs correct"
    by (rule CD_solvable_imp_BlackBox_solvable[OF CD_solv])

  — Step 2: BlackBox solvable yields False, via the bridge and FLP.
  have BB_unsolv:
    "¬ BlackBox_solvable procs correct"
    by (rule BlackBox_unsolvable_via_bridge[OF bridge])

  from BB_solv BB_unsolv show False by contradiction
qed
```

25 Theorem 4: CD impossible under broadcast

Paper, Theorem 4 (Section 4.2):

“It is impossible to solve causality determination (Definition 5) as specified by $\text{CD}(\mathbf{E}, \mathbf{F}, \mathbf{e}_i^*)$ in an asynchronous broadcast-based message passing system with one or more Byzantine processes.”

The paper’s proof of Theorem 4 “has the overall structure along the lines of that for Theorem 3”. Its two differences:

1. “By doing broadcasts using the Byzantine Reliable Broadcast (BRB) [...] layer, false positives can be prevented by ensuring no fake events/causal dependencies are added to $\mathbf{F}$.”
2. “False negatives still cannot be prevented (Theorem 1 carries over).”

Deviation: we do not formalise BRB. We use the same chain as Theorem 3, exploiting the mode-agnosticism of `CD_solvable` – the reduction-to-FLP works mode-independently. A richer development that adds BRB would refine our `Broadcast` predicate and re-state Theorem 4 with BRB as a hypothesis, but the *conclusion* – “CD is unsolvable” – is identical.

```

theorem CD_impossible_broadcast:
  assumes byz_ne: "byzantine  $\neq$  {}"
    and cor_ne: "correct  $\neq$  {}"
    and bridge: "bb_realizes_flp_consensus
                  procs correct TYPE('s) TYPE('v)"
  shows " $\neg$  CD_solvable Broadcast correct"
proof
  assume CD_solv: "CD_solvable Broadcast correct"
  have BB_solv: "BlackBox_solvable procs correct"
    by (rule CD_solvable_imp_BlackBox_solvable[OF CD_solv])
  have BB_unsolv: " $\neg$  BlackBox_solvable procs correct"
    by (rule BlackBox_unsolvable_via_bridge[OF bridge])
  from BB_solv BB_unsolv show False by contradiction
qed

```

26 Theorem 5: CD impossible under multicast

Paper, Theorem 5 (Section 4.2):

“It is impossible to solve causality determination (Definition 5) as specified by $\text{CD}(\mathbf{E}, \mathbf{F}, \mathbf{e}_i^*)$ in an asynchronous multicast-based message passing system with one or more Byzantine processes.

Proof. Unicast mode of communication is a special case of multicast where each group is of size 1 (or 2 if the sender is included in the multicast group). Theorem 3 proved that causality determination in the presence of even a single Byzantine process under unicast communication is impossible to solve. As the special case of group size 1 (or 2) is not solvable, the general case of multicast is also not solvable.”

Faithfulness: our `CD_solvable` predicate is mode-agnostic at the level of Definition 5 (see comment in `CD.thy`); a multicast algorithm could in particular be used as a unicast algorithm by specialising to a group of size 1. We reduce Theorem 5 to Theorem 3 explicitly via the function-level argument: if some `produces_valid_F` algorithm exists for Multicast, the same algorithm witnesses `CD_solvable Unicast`.

```

theorem CD_impossible_multicast:
  assumes byz_ne: "byzantine  $\neq$  {}"
    and cor_ne: "correct  $\neq$  {}"
    and bridge: "bb_realizes_flp_consensus
                  procs correct TYPE('s) TYPE('v)"
  shows " $\neg$  CD_solvable Multicast correct"
proof
  assume CD_solv: "CD_solvable Multicast correct"

```

— Specialise the multicast solver to the unicast mode. At the abstraction level of Definition 5 this is trivial: any algorithm that produces a valid F under multicasting also produces a valid F under unicasting (the validity predicate is mode-agnostic). Hence CD_solvable of **Multicast** entails the same for **Unicast**.

```

have unicast_solv: "CD_solvable Unicast correct"
proof -
  obtain alg where "produces_valid_F correct alg"
    using CD_solv by (auto simp: CD_solvable_def)
  thus ?thesis by (auto simp: CD_solvable_def)
qed

show False
  using CD_impossible_unicast[OF byz_ne cor_ne bridge] unicast_solv
  by contradiction
qed

```

27 Summary corollary

One statement, all three modes.

```

theorem CD_impossible_all_modes:
  assumes byz_ne: "byzantine  $\neq \{\}$ "
    and cor_ne: "correct  $\neq \{\}$ "
    and bridge: "bb_realizes_flp_consensus
      procs correct TYPE('s) TYPE('v)"
  shows " $\neg$  CD_solvable Unicast correct"
    and " $\neg$  CD_solvable Broadcast correct"
    and " $\neg$  CD_solvable Multicast correct"
proof -
  show " $\neg$  CD_solvable Unicast correct"
    by (rule CD_impossible_unicast[OF byz_ne cor_ne bridge])
  show " $\neg$  CD_solvable Broadcast correct"
    by (rule CD_impossible_broadcast[OF byz_ne cor_ne bridge])
  show " $\neg$  CD_solvable Multicast correct"
    by (rule CD_impossible_multicast[OF byz_ne cor_ne bridge])
qed

end — context byzantineSystem_with_identification

end

```

```

theory Foundation_Vacuity
  imports ByzantineSystem
begin

```

28 A pure-HOL “consensus solver” satisfying solves_Consensus

The abstract predicate solves_Consensus demands only Agreement and Validity on a HOL function; it places *no* constraint making the function implementable by an asynchronous distributed protocol. At this abstraction level the function below already satis-

fies the predicate, so any axiom of the shape “no abstract alg satisfies solves_Consensus” is unsatisfiable.

```
definition simple_alg :: "'p set  $\Rightarrow$  'p consensus_alg" where
  "simple_alg C V p  $\equiv$  ( $\exists q \in C. V q$ )"
```

```
lemma simple_alg_solves_Consensus:
  "solves_Consensus C (simple_alg C)"
proof -
  have AGR: " $\forall V. \text{consensus\_agreement } C \text{ (simple\_alg } C) V$ "
    by (auto simp: consensus_agreement_def simple_alg_def)
  have VAL: " $\forall V. \text{consensus\_validity } C \text{ (simple\_alg } C) V$ "
    by (auto simp: consensus_validity_def simple_alg_def)
  show ?thesis
    using AGR VAL by (simp add: solves_Consensus_def)
qed
```

```
lemma exists_consensus_alg:
  " $\exists \text{alg. solves\_Consensus } (C::'p \text{ set}) \text{ alg}$ "
  using simple_alg_solves_Consensus by blast
```

29 The natural-looking “abstract-consensus is unsolvable” claim is false

This is what the retired axiom denied. The negation is a HOL theorem, hence the original axiom was unsatisfiable.

```
lemma exists_abstract_consensus_solver:
  shows " $\neg (\neg (\exists \text{alg. solves\_Consensus } (C::'p \text{ set}) \text{ alg}))$ "
  using exists_consensus_alg by blast

end
```

References

- [1] B. Bisping, P.-D. Brodmann, T. Jungnickel, C. Rickmann, H. Seidler, A. Stüber, A. Wilhelm-Weidner, K. Peters, and U. Nestmann. A constructive proof for FLP. *Archive of Formal Proofs*, 2016. Formal proof development.
- [2] M. J. Fischer, N. A. Lynch, and M. S. Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM*, 32(2):374–382, 1985.
- [3] L. Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.
- [4] A. Misra and A. D. Kshemkalyani. Byzantine-tolerant detection of causality: There is no holy grail. *Parallel Computing*, 124:103136, 2025.